



武漢大學

WUHAN UNIVERSITY

第7讲 集成学习





武汉大学
WUHAN UNIVERSITY

01 个体与集成

02 Boosting

03 Bagging与随机森林

04 多样性



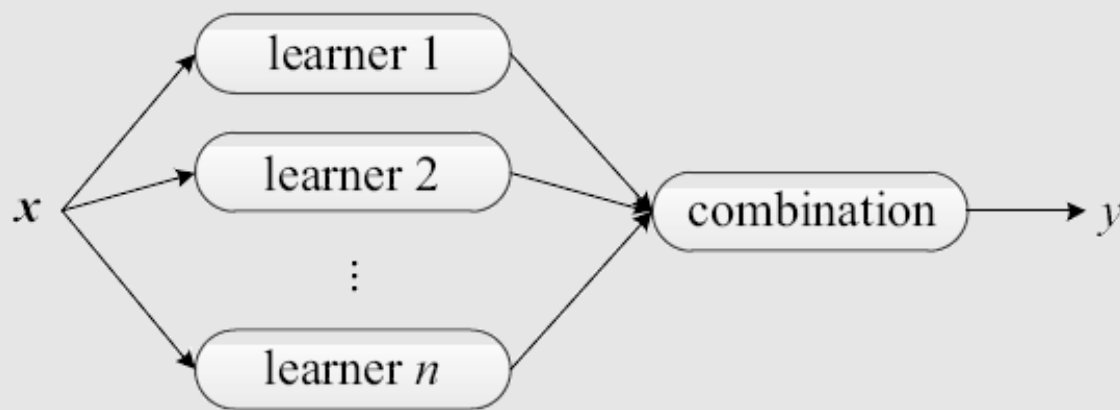
武汉大学
WUHAN UNIVERSITY

第一节 个体与集成

Individual and integration

- **集成学习 (Ensemble Learning)** 通过**构建并结合**多个学习器来完成学习任务，获得比单一学习器**更好的性能**，提高学习器的**泛化能力**。有时被称为**多分类器系统** (multi-classifier system)、**基于委员会的学习** (committee-based learning)

集成学习：集成由多个**个体学习器**构成，个体学习器分别在训练数据上训练获得，它们可以是决策树、神经网络或其他算法（**同质或异质**集成）





1 基学习器

- 同质集成 (Homogeneous Ensemble) : 集成中只包含**同种类型**的个体学习器, 此时个体学习器也被称为 “**基学习器 (base learner)**” , 相应的学习算法称为 “**基学习算法**”
- 异质集成 (Heterogeneous Ensemble) : 只要是使用**多个学习器**来解决问题, 就可以认为符合集成学习的广义形式。个体学习器由**不同的学习算法生成**, 不存在 “**基学习算法**” 。相应地, 个体学习器一般不称为 “**基学习器**”



2 多专家组合

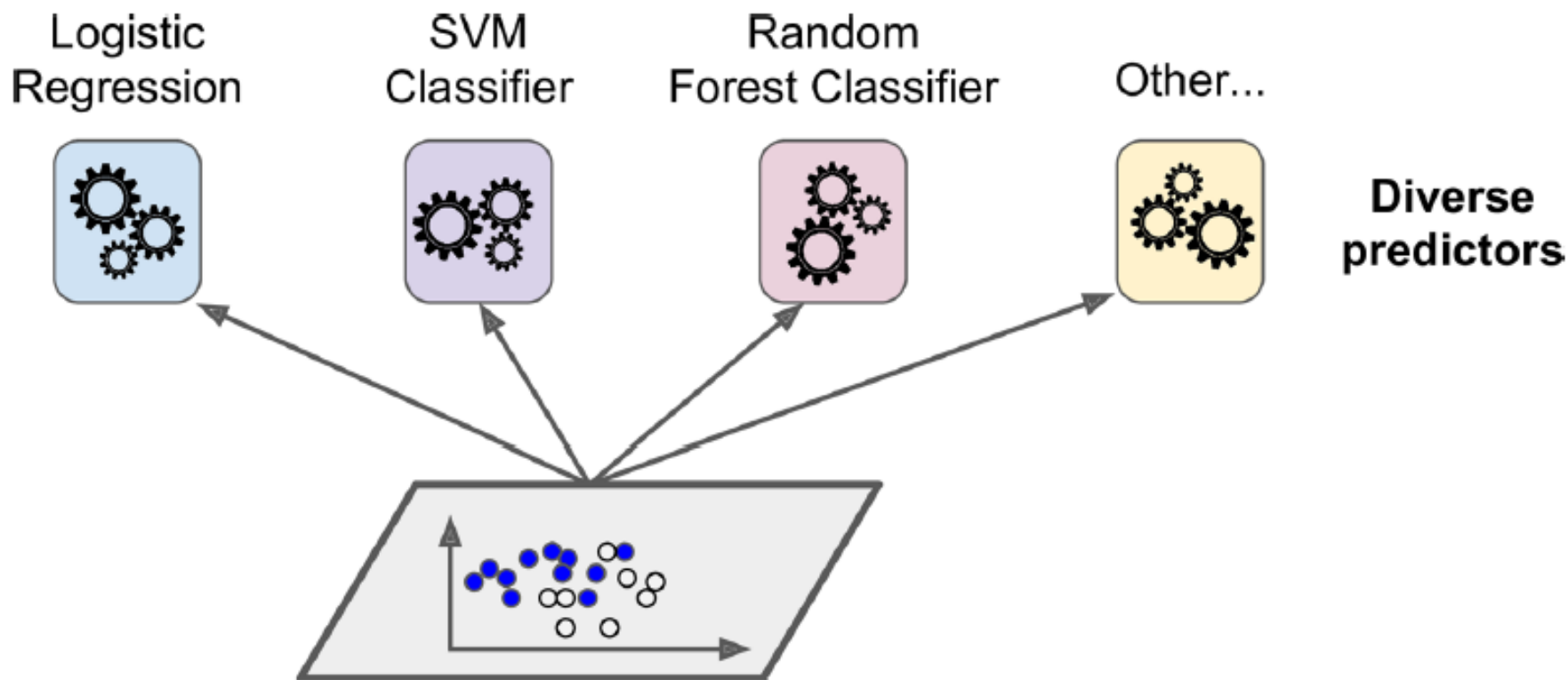
- 多专家组合 (multiexpert combination) 方法让个体学习器**并行**工作。这些方法可以进一步划分成两类:
 - 全局 (Global) 方法又称为**融合** (fusion) : 给定一个样本输入, **所有**的个体学习器都产生一个输出, 并且所有这些输出都要使用。通过平均、投票或堆叠来产生最终结果 → Bagging、Stacking
 - 局部 (Local) 方法又称为**选择** (selection) : 如混合专家(mixture of expert), 有一个门控模型动态考察输入, 并选择**一小部分**学习器来产生输出



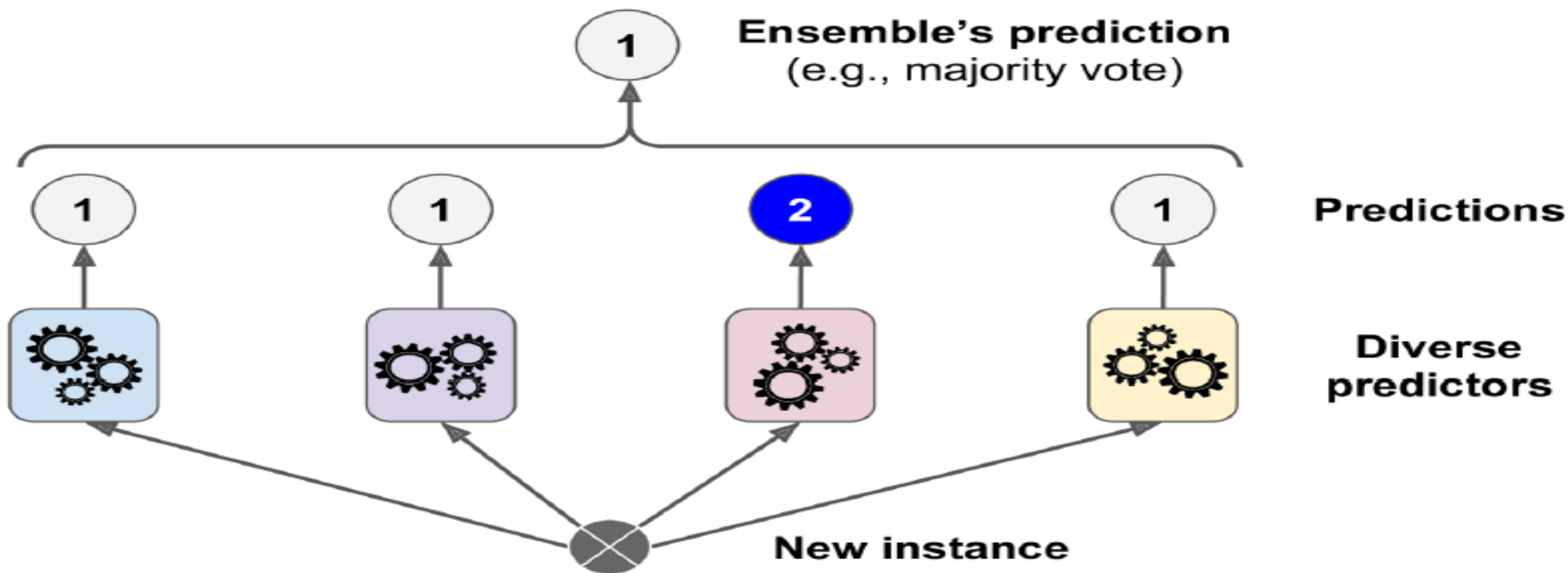
3 多级组合

- 多级组合 (multistage combination) 方法让个体学习器按复杂度递增排序, 构成一个**串行**的组合, 其中后一个基学习器在前一个基学习器预测不够准确的样本上进行训练, 或者对前一个学习器的预测残差、梯度进行调整 ← Boosting

- 假设已经训练好了多个分类器，每个分类器的准确率约为80%。这次分类器可以包括诸如：
一个逻辑回归分类器、一个SVM分类器、一个随机森林分类器（本身也是一种集成分类器）、一个K-近邻分类器，以及其他各种分类器。



- 要在这些已有分类器的基础上创建一个更好的分类器，最简单的办法就是**聚合**每个分类器的预测，然后将得票最多的结果作为预测类别。这种**简单多数投票分类器**被称为**硬投票分类器**。
- 这个投票分类器的准确率通常比其中任何一个单分类器还要好。甚至即使每个单分类器都是**弱学习器**（仅比随机猜测好一点），通过集成依然可以实现一个强学习器（高准确率），只要用于集成的弱学习器有**足够多的数量**、相互之间有**足够大的多样性和独立性**。



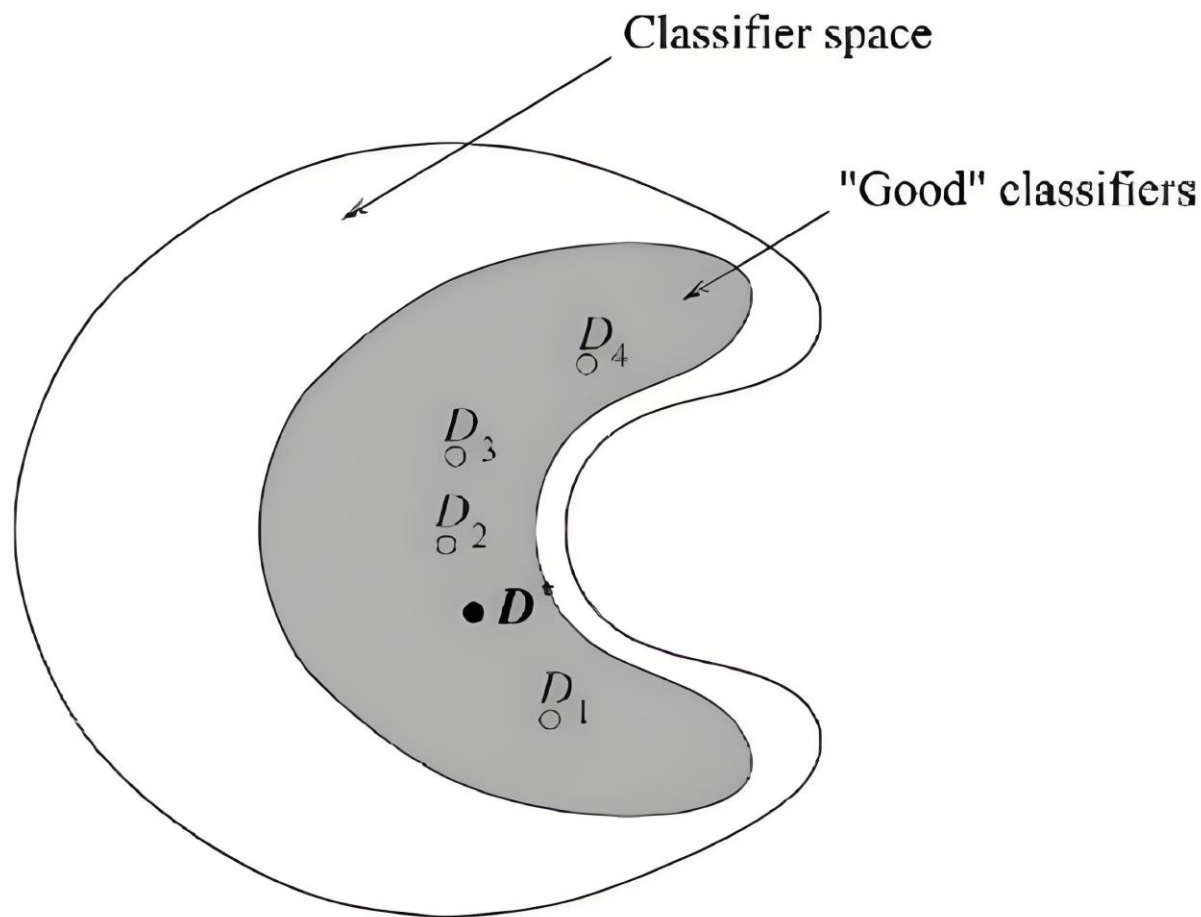


- 假设创建了一个包含1000 个分类器的集成，每个分类器都只有51% 的几率是正确的（几乎没比随机猜测强多少）。如果以**简单多数投票**的类别作为预测结果，你可以期待的准确率高达75%。

$$\sum_{k=501}^{1000} C_{1000}^k p^k (1-p)^{1000-k} \approx 0.75$$

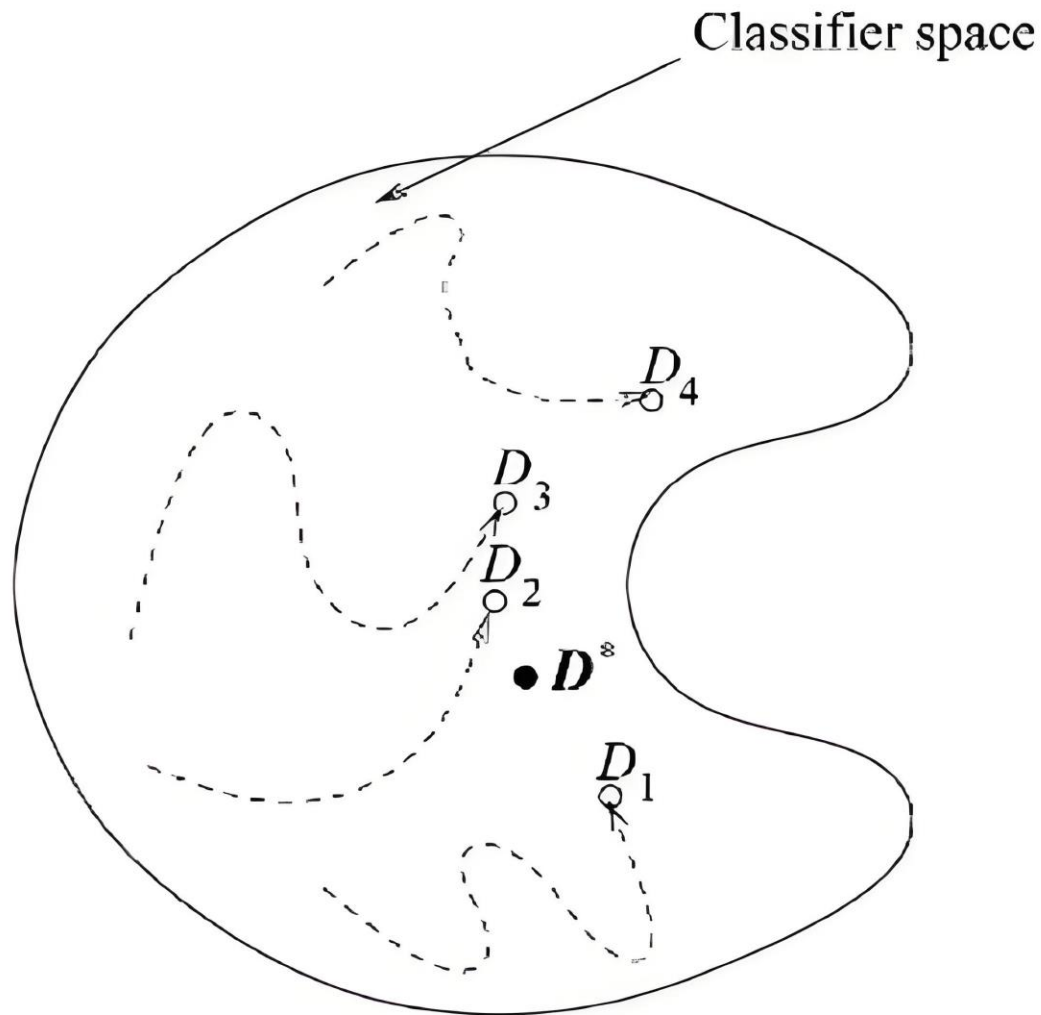
- 当然，能完全达到这个数字的前提是：所有的分类器都是完全**独立**的，彼此的错误毫不相关。显然这是不可能的，因为它们都是在相同的数据上训练的。它们很可能会犯相同的错误，导致集成的准确率有所降低
- 集成可以**提升性能、不起作用、起负作用（西瓜书上的例子）**
- 理想的集成：**好而不同（准确性与多样性本身存在冲突！）**

- 从统计分析方面看，虽然在同一训练集上可以学习训练性能最好的分类器，但是这些分类器在未来数据中的性能则是很难保证的。为了**降低这种单一选择的风险**，结合多学习器就是一种合理的选择。

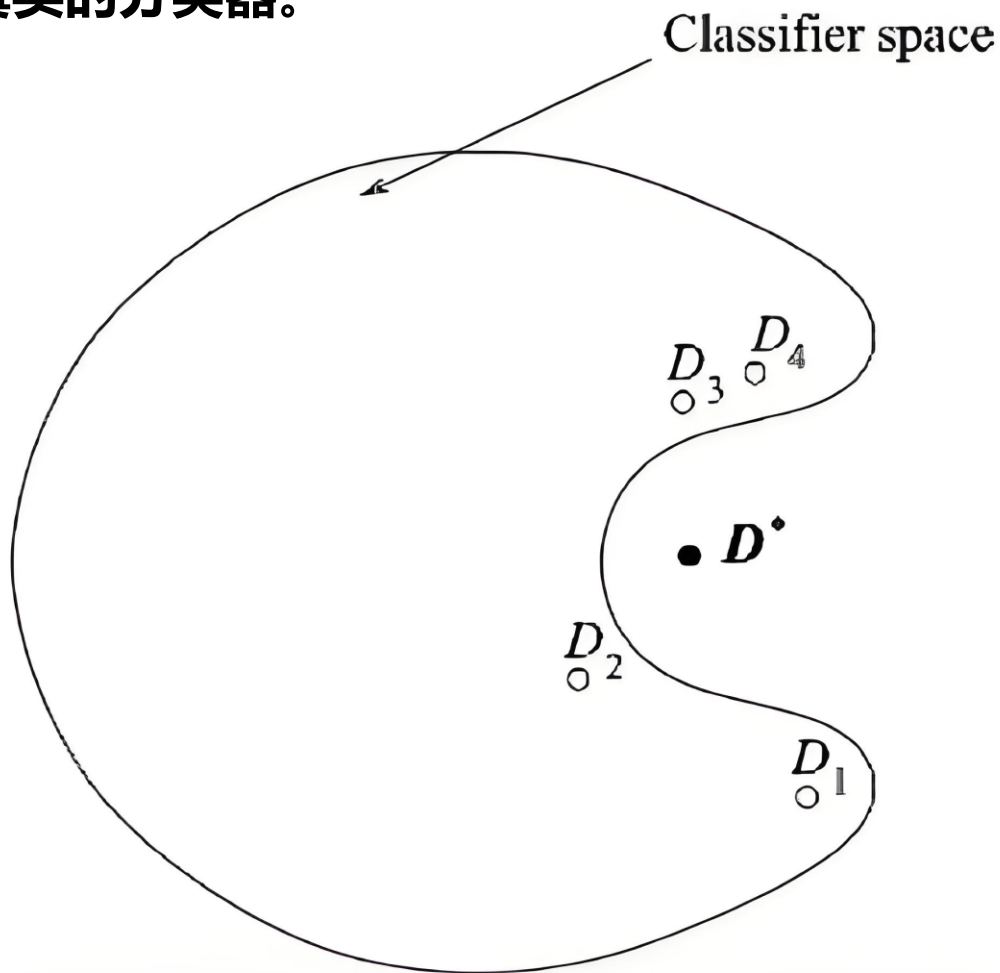




- 从算法求解方面看，很多学习算法的使用梯度下降法，往往会陷入到局部极小点，其泛化性能可能很低。通过分类器集成，可以**降低陷入局部极小点的风险**。

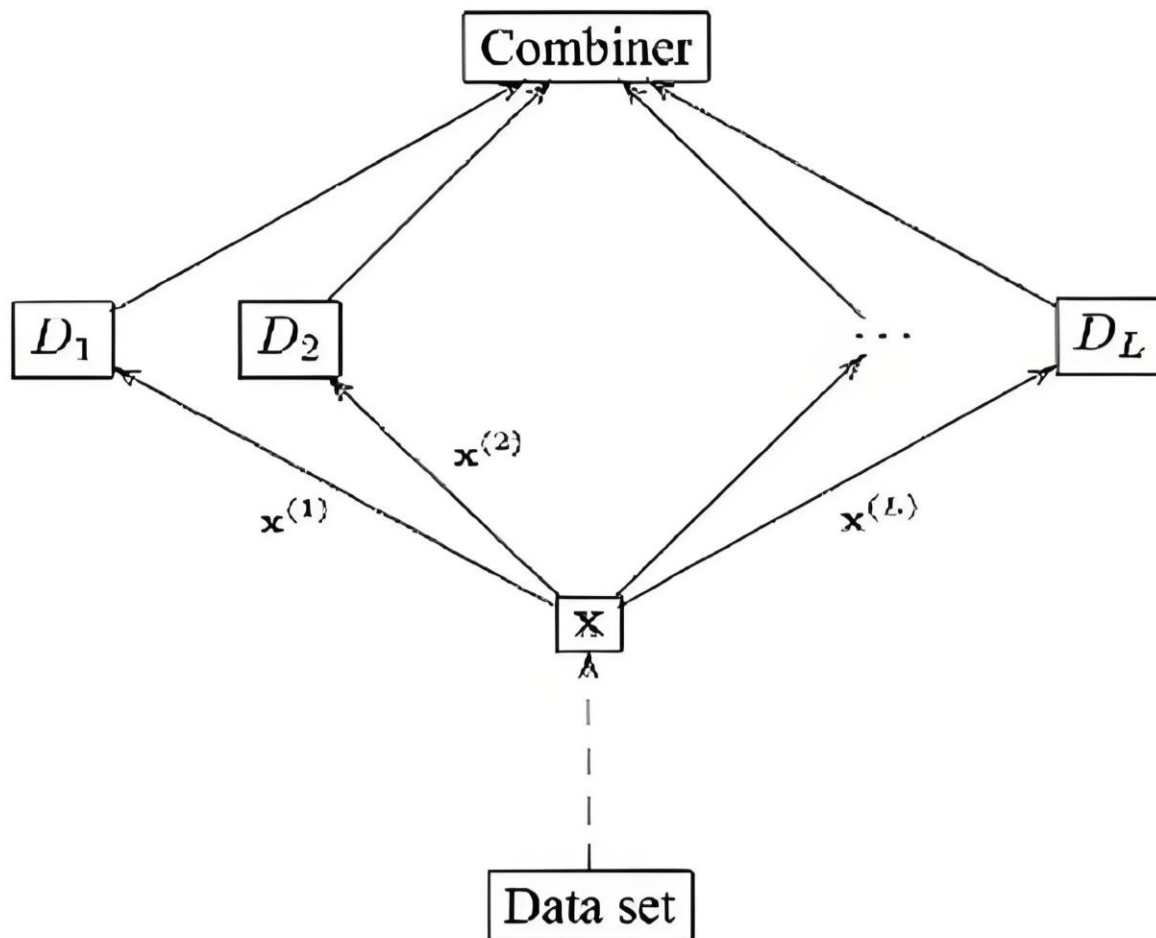


- 从模型表达方面看，某些学习任务的真实的最佳分类器可能不在当前学习算法所考虑的模型空间中，此时使用单学习器肯定是无效。通过集成多个学习器，比如分类器的线性组合，就有可能接近真实的分类器。





- 数据层面（自助抽样），特征层面（特征抽样）、模型层面（各种学习器）、集成层面（Boosting, Bagging, 输出融合、学习器选择）

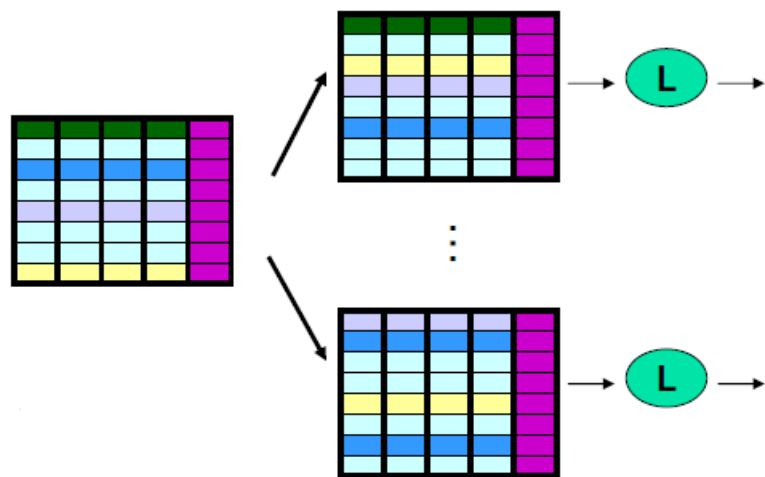


A. Combination level:
Design different
combiners.

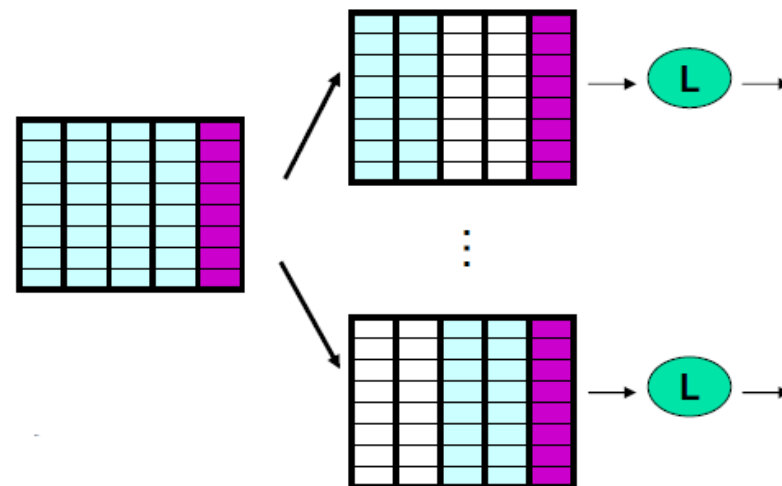
B. Classifier level:
Use different
base classifiers.

C. Feature level:
Use different
feature subsets.

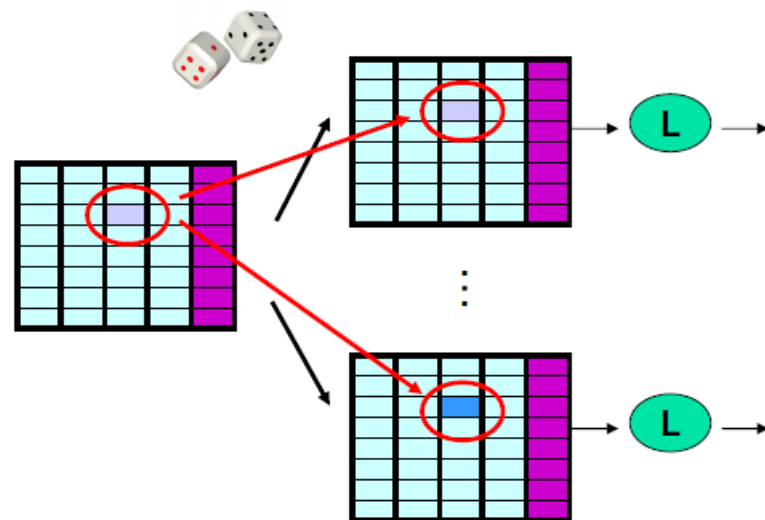
D. Data level:
Use different
data subsets.



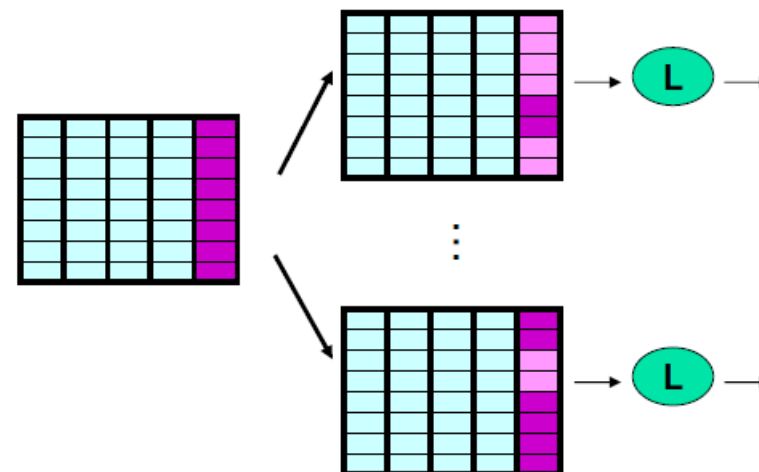
数据层面的多样化



特征层面的多样化



模型层面的多样化



集成层面的多样化



➤ 根据个体学习器的**生成方式**，集成学习方法大致可分为两大类：

➤ **迭代提升法**（Boosting）。提升法的总体思路是：循环训练个体学习器，每个**后继**学习器都对其**前驱**学习器做出一些改进。因此个体学习器之间存在**强依赖**关系，必须**串行**生成。代表方法有：**自适应提升法**（Adaptive Boosting, AdaBoost）和**梯度提升法**（Gradient Boosting）

➤ **自助聚合法**（Bagging）。自助聚合法中，每个个体学习器使用的算法是相同的，但是在不同的**自助样本集**上进行训练。因此个体学习器之间不存在强的计算依赖关系，可**并行**学习，最后再进行结果聚合。代表方法有：基于决策树的**随机森林法**（Random Forest）。由于可并行学习，自助聚合法尤其随机森林法得到广泛使用。

Bagging == Bootstrap **Aggregating**（装袋法）



武汉大学
WUHAN UNIVERSITY

02 Boosting

Boosting

《机器学习》

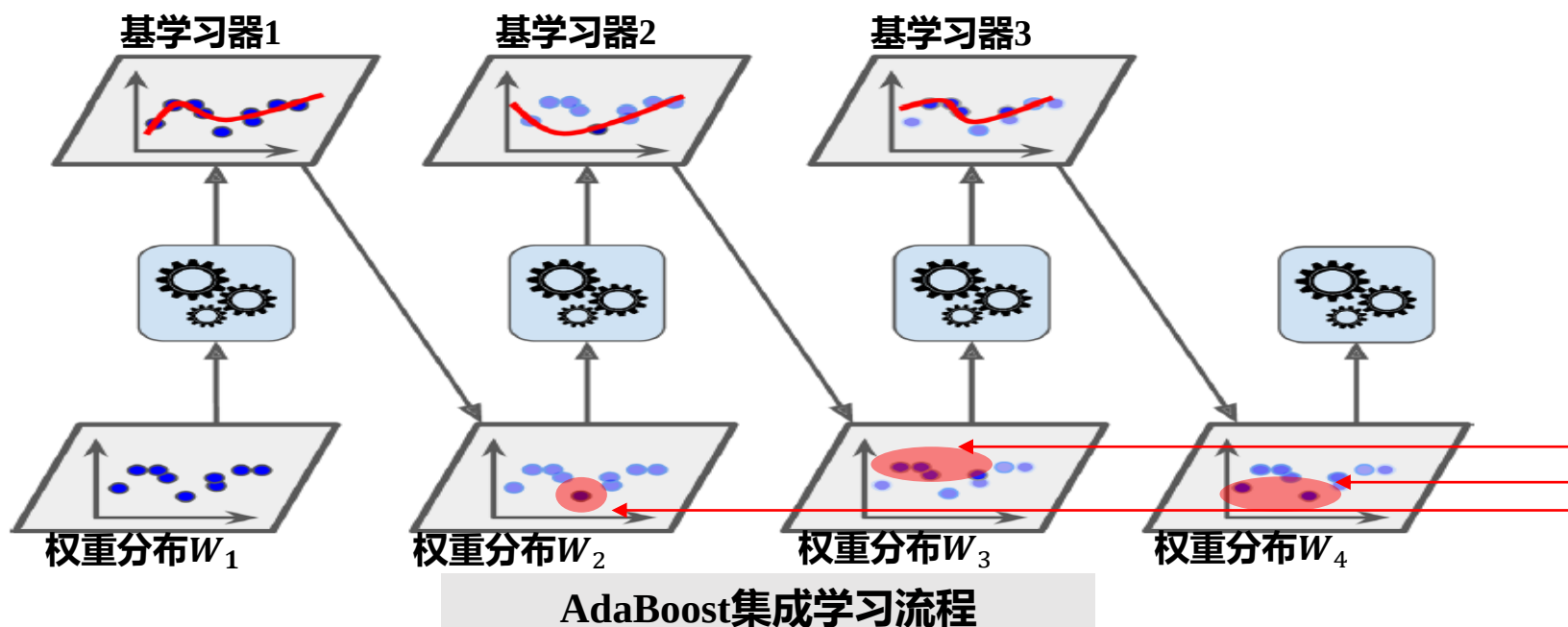


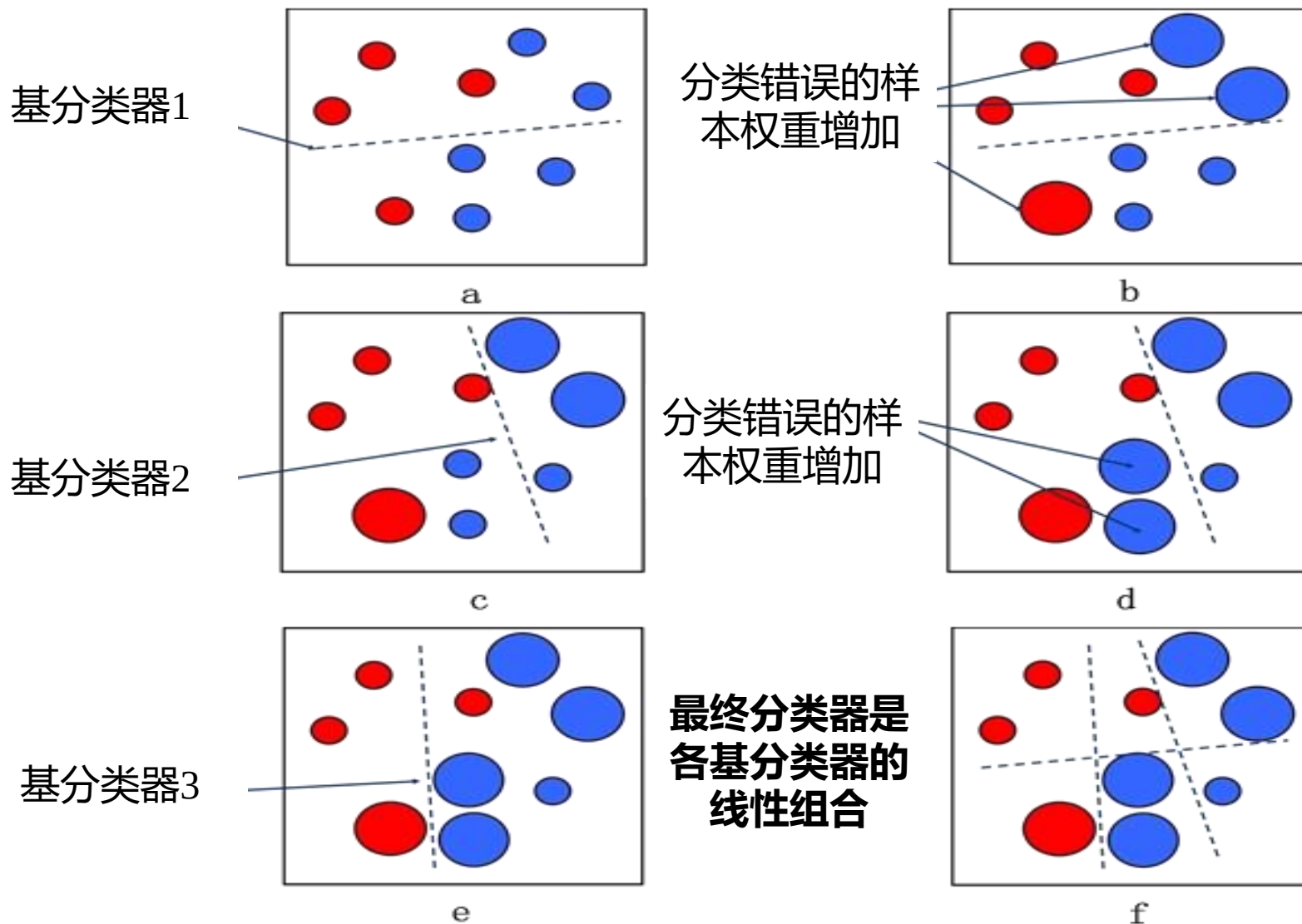
- Boosting是一类**串行**式集成学习方法。
- 它的基学习器之间存在**强依赖**关系。要求基学习器能对**特定的数据分布**进行学习，可通过“**重采样法**”和“**重赋权法**”来实施
- 代表方法为自适应提升法（Adaptive Boosting, AdaBoost）、梯度提升法（Gradient Boosting）等

- **重赋权方法** (re-weighting)。在训练过程的每一轮中，根据样本分布为每个训练样本重新赋予一个权重
- **重采样方法** (re-sampling)。对无法接受带权样本的基学习算法，可通过在每一轮学习中，根据样本分布对训练集重新采样，再用重采样获得的样本集对基学习器进行训练

1 AdaBoost方法

- 先从初始训练集训练出一个基学习器，根据基学习器的表现**对训练样本分布进行调整**，使得先前基学习器做错的训练样本在后续获得更多关注，基于调整分布的训练样本训练下一个基学习器；如此重复进行，直到基学习器的数目达到预先指定值 T ，最终将对 T 个基学习器进行**加权结合**





AdaBoost集成学习改变每一轮数据权重分布

➤ 给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ($x_i \in X$, $y_i \in \{-1, 1\}$)

和弱学习算法用AdaBoost构造最终的分类器 $G(x)$

注意正负类标签

AdaBoost流程

(1) 初始化训练数据的权值分布

$$W_1 = (w_{11}, \dots, w_{1i}, \dots, w_{1m}), \quad w_{1i} = \frac{1}{m}, \quad i = 1, 2, \dots, m$$

初始等权重

(2) 对 $t = 1, 2, \dots, T$

(a) 使用具有权值分布 W_t 的训练数据集学习, 得到基分类器

$$g_t(x): X \rightarrow \{-1, +1\}$$

(b) 计算 $g_t(x)$ 在训练数据集上的**分类错误率** e_t

$$e_t = \sum_{i=1}^m P(g_t(x_i) \neq y_i) = \sum_{i=1}^m w_{ti} I(g_t(x_i) \neq y_i)$$

即分类错误的样本的权重和



(c) 计算 $g_t(x)$ 的系数

$$\alpha_t = \frac{1}{2} \ln \frac{1 - e_t}{e_t}$$

分类错误率越小的基学习器，权重越大
分类错误率 >0.5 的基学习器权重为负（终止）

(d) 更新训练集的权值分布

$$W_{t+1} = (w_{t+1,1}, \dots, w_{t+1,i}, \dots, w_{t+1,m})$$

$$w_{t+1,i} = \frac{w_{ti}}{Z_t} \exp(-\alpha_t y_i g_t(x_i)), \quad i = 1, 2, \dots, m$$

分类正确：1
分类错误：-1

其中 Z_t 是一个规范化因子，使 W_{t+1} 成为一个概率分布

$$Z_t = \sum_{i=1}^m w_{ti} \exp(-\alpha_t y_i g_t(x_i))$$

(3) 构建基分类器的线性组合

$$f(x) = \sum_{t=1}^T \alpha_t g_t(x)$$

得到最终分类器：

$$G(x) = \underset{\uparrow}{\text{sign}}(f(x)) = \text{sign}\left(\sum_{t=1}^T \alpha_t g_t(x)\right)$$

组合学习器的符号决定最终分类



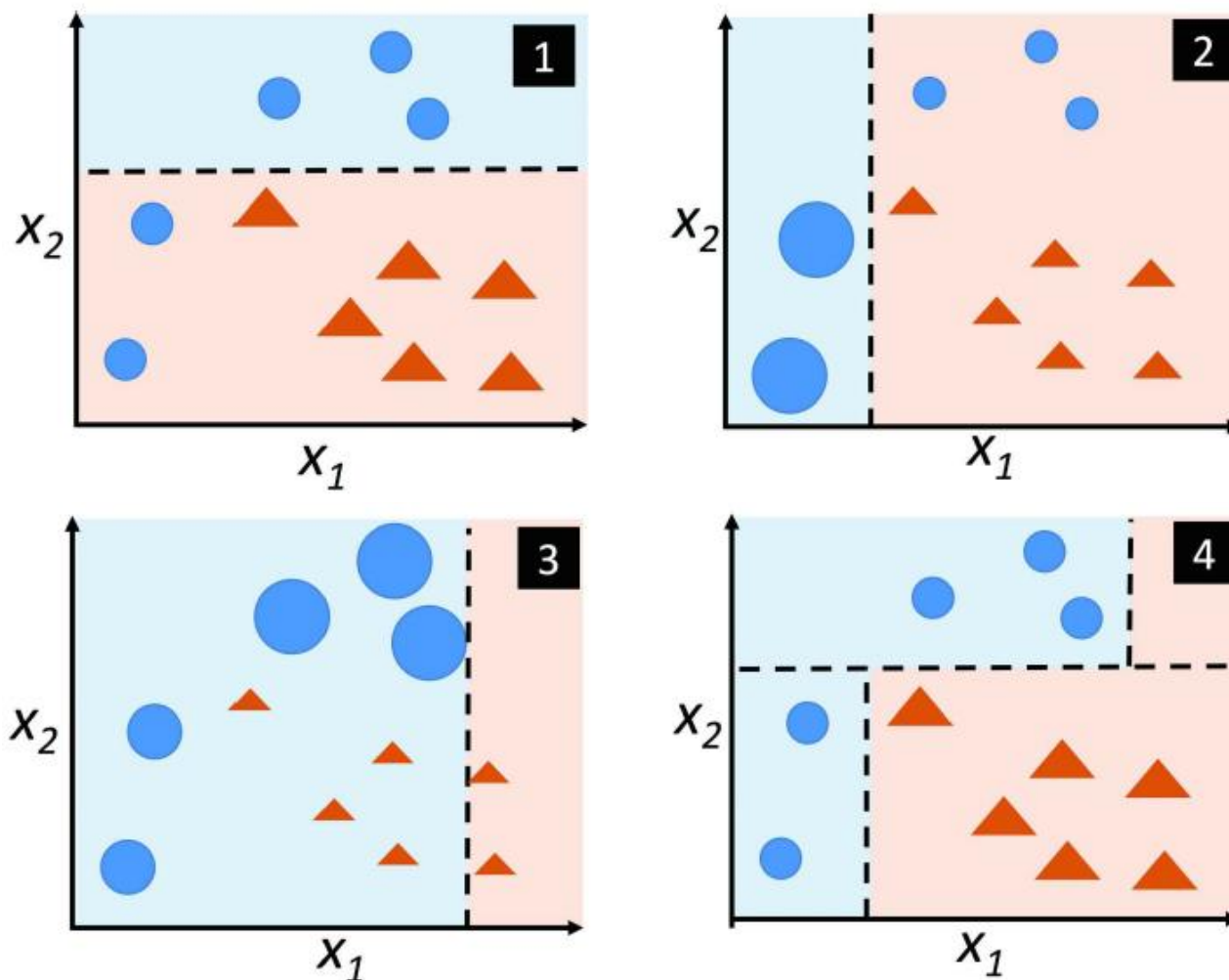
输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程:

```
1:  $\mathcal{D}_1(\mathbf{x}) = 1/m$ .  
2: for  $t = 1, 2, \dots, T$  do  
3:    $h_t = \mathcal{L}(D, \mathcal{D}_t)$ ;  
4:    $\epsilon_t = P_{\mathbf{x} \sim \mathcal{D}_t}(h_t(\mathbf{x}) \neq f(\mathbf{x}))$ ;  
5:   if  $\epsilon_t > 0.5$  then break  
6:    $\alpha_t = \frac{1}{2} \ln \left( \frac{1-\epsilon_t}{\epsilon_t} \right)$ ;  
7:    $\mathcal{D}_{t+1}(\mathbf{x}) = \frac{\mathcal{D}_t(\mathbf{x})}{Z_t} \times \begin{cases} \exp(-\alpha_t), & \text{if } h_t(\mathbf{x}) = f(\mathbf{x}) \\ \exp(\alpha_t), & \text{if } h_t(\mathbf{x}) \neq f(\mathbf{x}) \end{cases}$   
       $= \frac{\mathcal{D}_t(\mathbf{x}) \exp(-\alpha_t f(\mathbf{x}) h_t(\mathbf{x}))}{Z_t}$   
8: end for  
输出:  $H(\mathbf{x}) = \text{sign} \left( \sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$ 
```

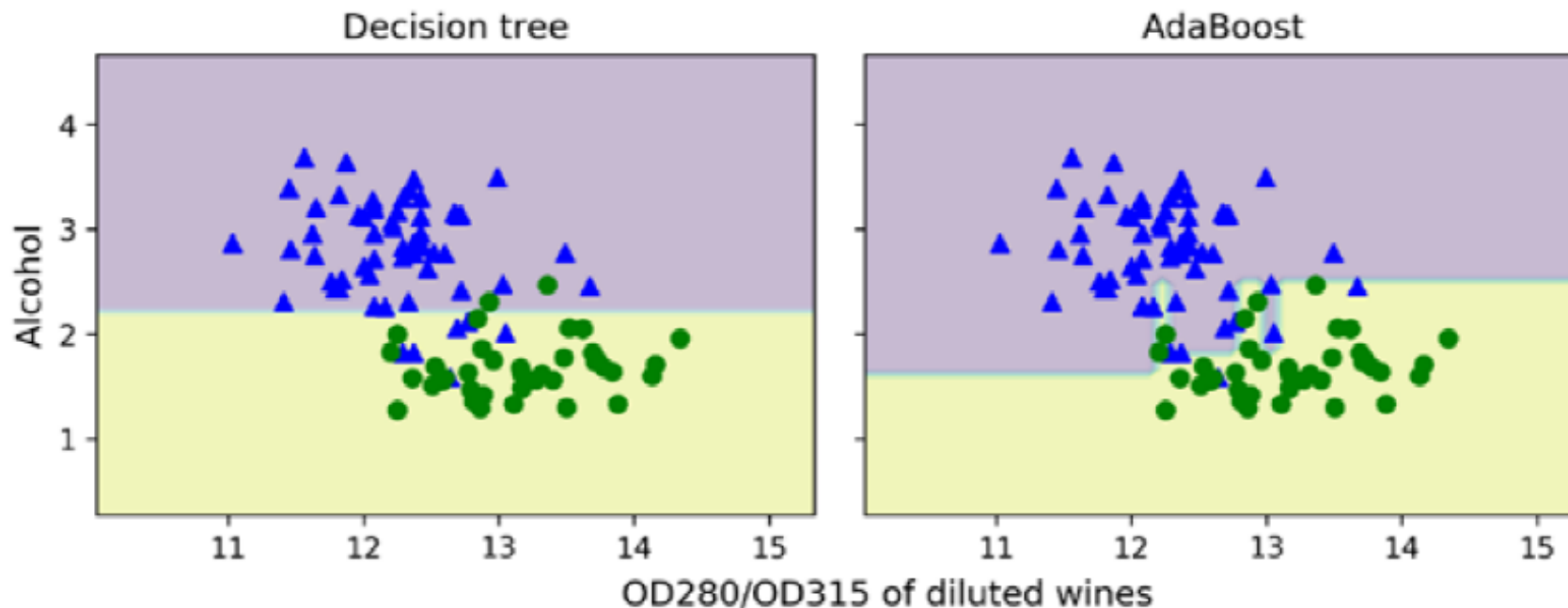
自适应提升 (AdaBoost) 算法流程图

上述算法的权重更新与分布更新的原理请阅读西瓜书p173-176



原始增强算法的原理示意图

图1、2、3表示串行学习的三个决策树的分类结果，
图4表示对三个分类器进行集成的结果



```
>>> from sklearn.ensemble import AdaBoostClassifier
>>> tree = DecisionTreeClassifier(criterion='entropy',
...                               random_state=1,
...                               max_depth=1)
>>> ada = AdaBoostClassifier(base_estimator=tree,
...                            n_estimators=500,
...                            learning_rate=0.1,
...                            random_state=1)
```

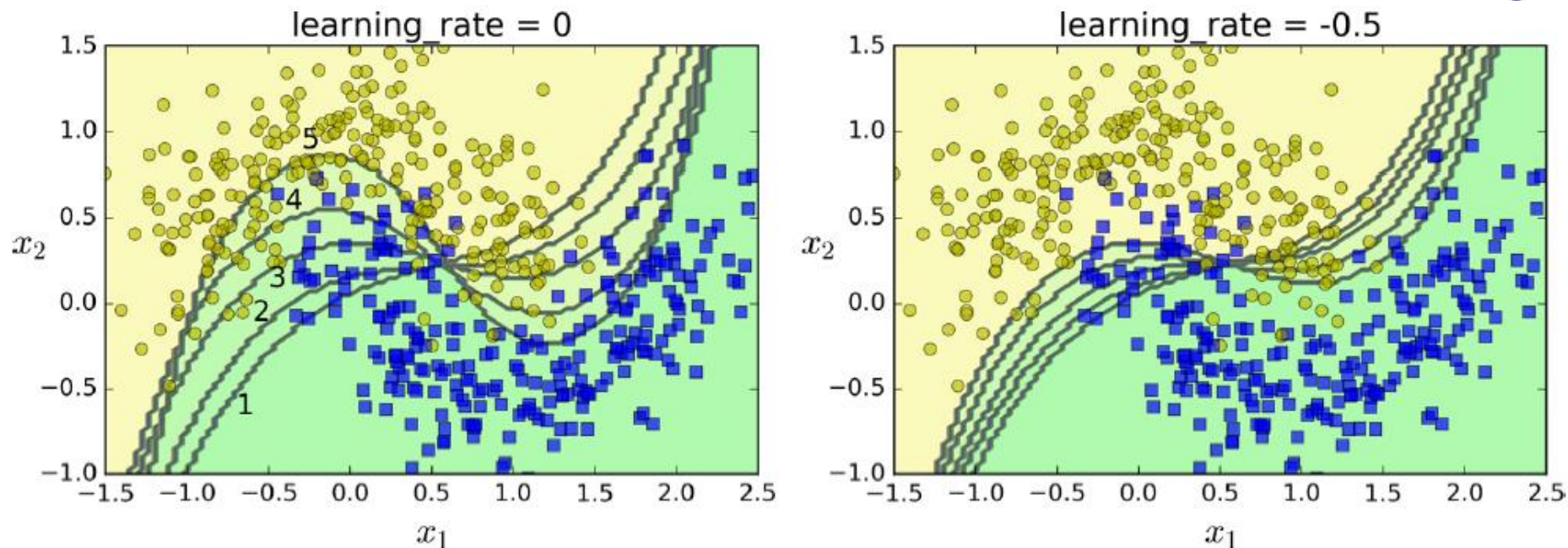


```
>>> tree = tree.fit(X_train, y_train)
>>> y_train_pred = tree.predict(X_train)
>>> y_test_pred = tree.predict(X_test)
>>> tree_train = accuracy_score(y_train, y_train_pred)
>>> tree_test = accuracy_score(y_test, y_test_pred)
>>> print('Decision tree train/test accuracies %.3f/%.3f'
...       % (tree_train, tree_test))
Decision tree train/test accuracies 0.916/0.875
```

单层决策树似乎对训练数据欠拟合：因为训练准确度和测试准确度都不太高

```
>>> ada = ada.fit(X_train, y_train)
>>> y_train_pred = ada.predict(X_train)
>>> y_test_pred = ada.predict(X_test)
>>> ada_train = accuracy_score(y_train, y_train_pred)
>>> ada_test = accuracy_score(y_test, y_test_pred)
>>> print('AdaBoost train/test accuracies %.3f/%.3f'
...       % (ada_train, ada_test))
AdaBoost train/test accuracies 1.000/0.917
```

AdaBoost正确地预测了训练集的所有分类标签，而且与单层决策树相比，其测试性能也略有改善 (0.917)



AdaBoost算法中串行优化的学习器的决策边界

```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=200,
    algorithm="SAMME.R", learning_rate=0.5
)
ada_clf.fit(X_train, y_train)
```



例题：根据下列样本，获得一个强分类器

X	0	1	2	3	4	5
Y	1	1	-1	-1	1	-1

w_1 初始权重0.167

● 第一轮提升

1、选择最优弱分类器

若按**1.5**切分数据，得弱分类器 $g_1(x)$: $x < 1.5$, 则 $y = 1$; $x > 1.5$, 则 $y = -1$

此时错误率 e_1 为 $1 \times 0.167 = 0.167$ (**分类错误的样本权重之和**)

2、计算最优弱分类器的权重

$$a_1 = 0.5 \times \ln((1 - 0.167) / 0.167) = 0.8047$$

3、更新样本权重

$x = 0, 1, 2, 3, 5$ 时, y 分类正确, 则样本权重为: $0.167 \times \exp(-0.8047) = 0.075$

$x = 4$ 时, y 分类错误, 则样本权重为: $0.167 \times \exp(0.8047) = 0.373$

规范化: $0.075 / 0.748 = 0.10$, $0.373 / 0.748 = 0.50$

新的样本权重 w_2 : $(0.1, 0.1, 0.1, 0.1, 0.5, 0.1)$

第一轮迭代后构造的分类器: $G(x) = 0.8047 \times g_1(x)$

分类器的错误率为 $1 / 6 = 0.167$

● 第二轮提升

1、选择最优弱分类器

若按**4.5**切分数据, 得弱分类器 $g_2(x)$: $x < 4.5$, 则 $y = 1$; $x > 4.5$, 则 $y = -1$

此时错误率 e_2 为 $2 \times 0.1 = 0.2$

x	0	1	2	3	4	5
y	1	1	-1	-1	1	-1
w	0.1	0.1	0.1	0.1	0.5	0.1

2、计算最优弱分类器的权重

$$a_2 = 0.5 \times \ln((1 - 0.2) / 0.2) = 0.6931$$

3、更新样本权重

$x = 0, 1, 5$ 时, y 分类正确, 则样本权重为: $0.1 \times \exp(-0.6931) = 0.05$

$x = 4$ 时, y 分类正确, 则样本权重为: $0.5 \times \exp(-0.6931) = 0.25$

$x = 2, 3$ 时, y 分类错误, 则样本权重为: $0.1 \times \exp(0.6931) = 0.2$

规范化后新的样本权重 w_3 : (0.0625, 0.0625, 0.250, 0.250, 0.3125, 0.0625)

第二轮迭代后构造的分类器: $G(x) = 0.8047 \cdot g_1(x) + 0.6931 \cdot g_2(x)$



● 第三轮提升

1、选择最优弱分类器

若按3.5切分数据，得弱分类器 $g_3(x)$: $x > 3.5$, 则 $y = 1$; $x < 3.5$, 则 $y = -1$ 此时错误率 e_3 为 $3 \times 0.0625 = 0.1875$

x	0	1	2	3	4	5
y	1	1	-1	-1	1	-1
w	0.0625	0.0625	0.25	0.25	0.3125	0.0625

注意这里符号和前两轮不一样！

2、计算最优弱分类器的权重

$$a_3 = 0.5 \times \ln((1 - 0.1875) / 0.1875) = 0.7332$$

3、更新样本权重

$x = 2, 3$ 时, y 分类正确, 则样本权重为: $0.25 \times \exp(-0.7332) = 0.1201$

$x = 4$ 时, y 分类正确, 则样本权重为: $0.3125 \times \exp(-0.7332) = 0.1501$

$x = 0, 1, 5$ 时, y 分类错误, 则样本权重为: $0.0625 \times \exp(0.7332) = 0.1301$

规范化后新的样本权重为(0.1667, 0.1667, 0.1539, 0.1539, 0.1923, 0.1667)

提问：AdaBoost算法中的Adaptiveness自适应体现在哪？

第三轮迭代后构造的分类器： $G(x) = 0.8047 \cdot g_1(x) + 0.6931 \cdot g_2(x) + 0.7332 \cdot g_3(x)$

按 $G(x)$ 分类所有样本均分类正确，强分类器的**错误率为 0**，停止迭代

$$\begin{cases} g_1(x): x < 1.5, \text{则 } y = 1; x > 1.5, \text{则 } y = -1 \\ g_2(x): x < 4.5, \text{则 } y = 1; x > 4.5, \text{则 } y = -1 \\ g_3(x): x > 3.5, \text{则 } y = 1; x < 3.5, \text{则 } y = -1 \end{cases}$$

X	0	1	2	3	4	5	
Y	1	1	-1	-1	1	-1	
g_1	1	1	-1	-1	-1	-1	$\times 0.8047$
g_2	1	1	1	1	1	-1	$\times 0.6931$
g_3	-1	-1	-1	-1	1	1	$\times 0.7332$
$G(x)$	1	1	-1	-1	1	-1	

- AdaBoost方法的核心在于每轮**提升**都对样本的**权重进行更新**，其权重更新公式可以进一步写为：

$$w_{t+1,i} = \begin{cases} \frac{w_{ti}}{Z_t} e^{-\alpha_t} & , y_i g_t(x_i) = 1 \quad \text{分类正确} \\ \frac{w_{ti}}{Z_t} e^{\alpha_t} & , y_i g_t(x_i) = -1 \quad \text{分类不正确} \end{cases}$$

作业：试推导样本 x_i 第 t 轮的权重与第 $t+1$ 轮权重存在如下迭代关系

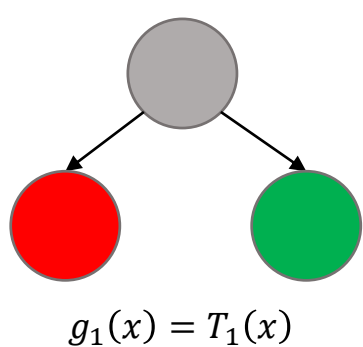
$$w_{t+1,i} = \begin{cases} \frac{w_{ti}}{2(1 - e_t)} & \text{分类正确} \\ \frac{w_{ti}}{2e_t} & \text{分类不正确} \end{cases}$$

2 梯度提升树

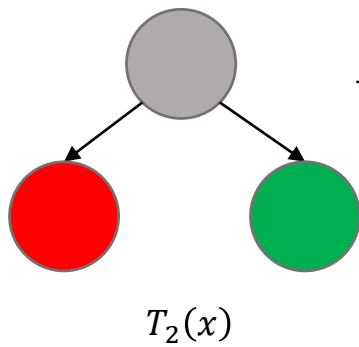
- 以**决策树**方法构建基学习器的**Boosting**方法称为 “**提升树**”
- 以**CART**为基学习器，可构造用于分类/回归问题的**梯度提升树**

- 将Adaboost方法中 α_t 均设为1， $g_t(x)$ 视为一棵棵分类二叉树：

$$G(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t g_t(x) \right)$$



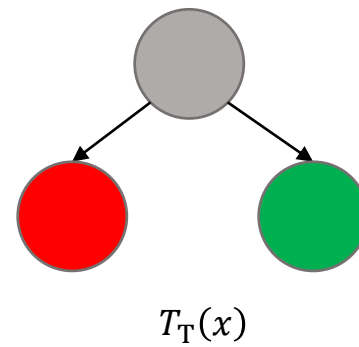
+



+

...

+



提升树模型



e.g 一维样本的**提升树回归**

x_i	1	2	3	4	5	6	7	8	9	10
y_i	5.56	5.7	5.91	6.4	6.8	7.05	8.9	8.7	9	9.05



《统计学习方法》P168

- **第一轮提升** • 令 $y_i = r_{1i}$ 寻找最优划分点 s

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
c_1	5.56	5.63	5.72	5.89	6.07	6.24	6.62	6.88	7.11
c_2	7.5	7.73	7.99	8.25	8.54	8.91	8.92	9.03	9.05
L(s)	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

- 划分区域为: $R1=\{1,2,3,4,5,6\}, R2=\{7,8,9,10\}$
- 对应**输出值**: $\hat{c}_1 = 6.24, \hat{c}_2 = 8.91$ 代价函数最小, 最优划分

$$a^*, s^* = \underset{a, s}{\operatorname{argmin}} [\sum_{x_i \in R_1(a, s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(a, s)} (y_i - \hat{c}_2)^2] = \underset{a, s}{\operatorname{argmin}} L(s)$$

- 其中 $\hat{c}_1 = \frac{1}{N_1} \sum_{x_i \in R_1(a, s)} y_i, \hat{c}_2 = \frac{1}{N_2} \sum_{x_i \in R_2(a, s)} y_i$

a 为最优划分特征, s 为划分属性。对于一维样本只关心 s

- 得到回归树 $T_1(x)$ 为: $T_1(x) = \begin{cases} 6.24 & x < 6.5 \\ 8.91 & x \geq 6.5 \end{cases}$, 令 $g_1(x) = T_1(x)$

- $g_1(x)$ 的回归预测值与数据标记值的残差:

注意: 这里开始与最小二乘回归树不一样了

x_i	1	2	3	4	5	6	7	8	9	10
y_i	5.56	5.7	5.91	6.4	6.8	7.05	8.9	8.7	9	9.05
$g_1(x_i)$	6.24	6.24	6.24	6.24	6.24	6.24	8.91	8.91	8.91	8.91
r_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

- 回归的总体损失取平方误差计算:

$$Loss(y, g_1(x)) = \sum_{i=1}^{10} (y_i - g_1(x_i))^2 = 1.93$$

总体损失: 平方误差

● 第二轮提升

- 求回归树 $T_2(x)$, 仍采用最小二乘回归树方法, **样本数据改为残差数据 r_{2i}**

x_i	1	2	3	4	5	6	7	8	9	10
r_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
c_1	-0.68	-0.61	-0.52	-0.35	-0.17	0.00	0.00	-0.03	-0.02
c_2	0.07	0.15	0.22	0.23	0.16	0.00	0.01	0.12	0.14
$L(s)$	1.42	1.00	0.79	1.13	1.66	1.93	1.93	1.90	1.91

- 对应的输出值: $\hat{c}_1 = -0.52$, $\hat{c}_2 = 0.22$, 此时回归树

- 则学习器 $g_2(x) = g_1(x) + T_2(x) = \begin{cases} 5.72 & x < 3.5 \\ 6.46 & 3.5 \leq x < 6.5 \\ 9.13 & x \geq 6.5 \end{cases}$
- 继续计算 $g_2(x)$ 拟合的预测值与训练数据标记值的残差：

x_i	1	2	3	4	5	6	7	8	9	10
r_{3i}	-0.16	-0.02	0.19	-0.06	0.34	0.59	-0.23	-0.43	-0.13	-0.08

- 回归的总体损失：

$$Loss(y, g_2(x)) = \sum_{i=1}^{10} (y_i - g_2(x_i))^2 = 0.79$$



● 第三轮提升

- 继续在残差数据 r_3 中训练回归树 $T_3(x) = \begin{cases} 0.15 & x < 6.5 \\ -0.22 & x \geq 6.5 \end{cases}$
- 则学习器 $g_3(x) = g_2(x) + T_3(x) = \begin{cases} 5.87 & x < 3.5 \\ 6.61 & 3.5 \leq x < 6.5 \\ 8.91 & x \geq 6.5 \end{cases}$
- $g_3(x)$ 的回归预测值与数据标记值的残差:

x_i	1	2	3	4	5	6	7	8	9	10
r_{4i}	-0.31	-0.17	0.04	-0.24	0.19	0.44	-0.01	-0.21	0.09	0.14

$$Loss(y, g_3(x)) = \sum_{i=1}^{10} (y_i - g_3(x_i))^2 = 0.47$$

- 继续提升至第六轮...

- 提升树 $g_6(x) = g_5(x) + T_6(x) = T_1(x) + \dots + T_6(x) = \begin{cases} 5.63 & x < 2.5 \\ 5.82 & 2.5 \leq x < 3.5 \\ 6.56 & 3.5 \leq x < 4.5 \\ 6.83 & 4.5 \leq x < 6.5 \\ 8.95 & x \geq 6.5 \end{cases}$
- 此时数据残差

$$Loss(y, g_6(x)) = \sum_{i=1}^{10} (y_i - g_6(x_i))^2 = 0.17$$

回归残差足够小，停止提升，得到最终的**回归学习器** $G(x)$

$$G(x) = g_6(x)$$

➤ 给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ($x_i \in X$, $y_i \in \mathbf{R}$)

用提升树构造最终的分类器 $G(x)$

提升树流程

(1) 初始化 $g_0(x) = 0$

(2) 对 $t = 1, 2, \dots, T$,

(a) 计算标记值与**上一轮**预测值的残差

$$r_{ti} = y_i - g_{t-1}(x_i), i = 1, 2, 3, \dots, m$$

(b) 拟合残差 r_{ti} 训练一个回归树, 得到 $T_t(x; \theta_t)$

(c) 更新 $g_t(x) = g_{t-1}(x) + T_t(x; \theta_t)$

(3) 得到**回归提升树**:

$$G(x) = \sum_{t=1}^T T_t(x; \theta_t)$$

决策树的参数, 即其最
优划分和输出值



- 注意到，当损失函数定义为**平方误差**时，**损失函数关于模型预测值的导数恰好就是“残差”**。定义残差 r_t 的方向与损失函数负梯度的方向一致。因此通过拟合残差，新构建的学习器能够沿着梯度下降的方向来优化模型

$$r_{ti} = -\frac{\partial l(y_i, g_{t-1}(x_i))}{\partial g_{t-1}(x_i)} = y_i - g_{t-1}(x_i)$$

注：平方损失求导请自行推导



- 对于**一般损失函数**而言，不是去直接拟合“残差”，而是去**拟合损失函数关于模型预测值的负梯度**，这个负梯度又称为“**伪残差**”。使用代价函数的梯度信息来指导后续决策树的构建，以逐步优化模型，降低全局损失，这种提升树也被称为“**梯度提升树**（Gradient Boosting Decision Tree, GBDT）”
- GBDT不直接拟合残差而是拟合负梯度的原因主要包括：
 - **扩展**到其他损失函数：包括那些梯度与残差不相等的情况（如分类问题中的交叉熵损失函数或带有正则项的损失函数）
 - 负梯度方向是代价函数下降最快的方向，可以提高优化**效率**
 - 直接使用残差计算可能导致数值不**稳定**，拟合梯度更稳定

➤ 对于损失函数 $l(y_i, g_{t-1}(x_i))$ 有：

- 平方损失： $\frac{1}{2}(\hat{y}_i - y_i)^2$

导数为 $\hat{y}_i - y_i$ ，正好等于残差。拟合残差与拟合梯度一致

- 交叉熵损失： $-y_i \cdot \ln \hat{y}_i - (1 - y_i) \cdot \ln(1 - \hat{y}_i)$

当使用GBDT进行二分类时， $g_{t-1}(x_i)$ 为前 $t-1$ 棵决策树相加

注意与前面的区别 $\begin{cases} g_{t-1}(x_i) = \sum_{1}^{t-1} T_t(x_i; \theta_t) \\ \hat{y}_i = \frac{1}{1 + e^{-g_{t-1}(x_i)}} \end{cases}$ 满足 $\hat{y}_i' = \hat{y}_i(1 - \hat{y}_i)$ Logistic函数性质

$$\text{则: } \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i} = \frac{-y_i}{\hat{y}_i} + \frac{1 - y_i}{(1 - \hat{y}_i)}$$

$$\frac{\partial l(y_i, g_{t-1}(x_i))}{\partial g_{t-1}(x_i)} = \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial g_{t-1}(x_i)} = \left[\frac{-y_i}{\hat{y}_i} + \frac{1 - y_i}{(1 - \hat{y}_i)} \right] \cdot \hat{y}_i(1 - \hat{y}_i) = \hat{y}_i - y_i$$



- 要注意“梯度提升树”中的“梯度”与“梯度下降法”中的“梯度”的区别和联系
 - 梯度提升树：损失函数关于**模型预测值**的负梯度
 - 梯度下降法：损失函数关于**参数**的负梯度
- GBDT的计算流程与一般提升树类似（参见李航机器学习方法p144）





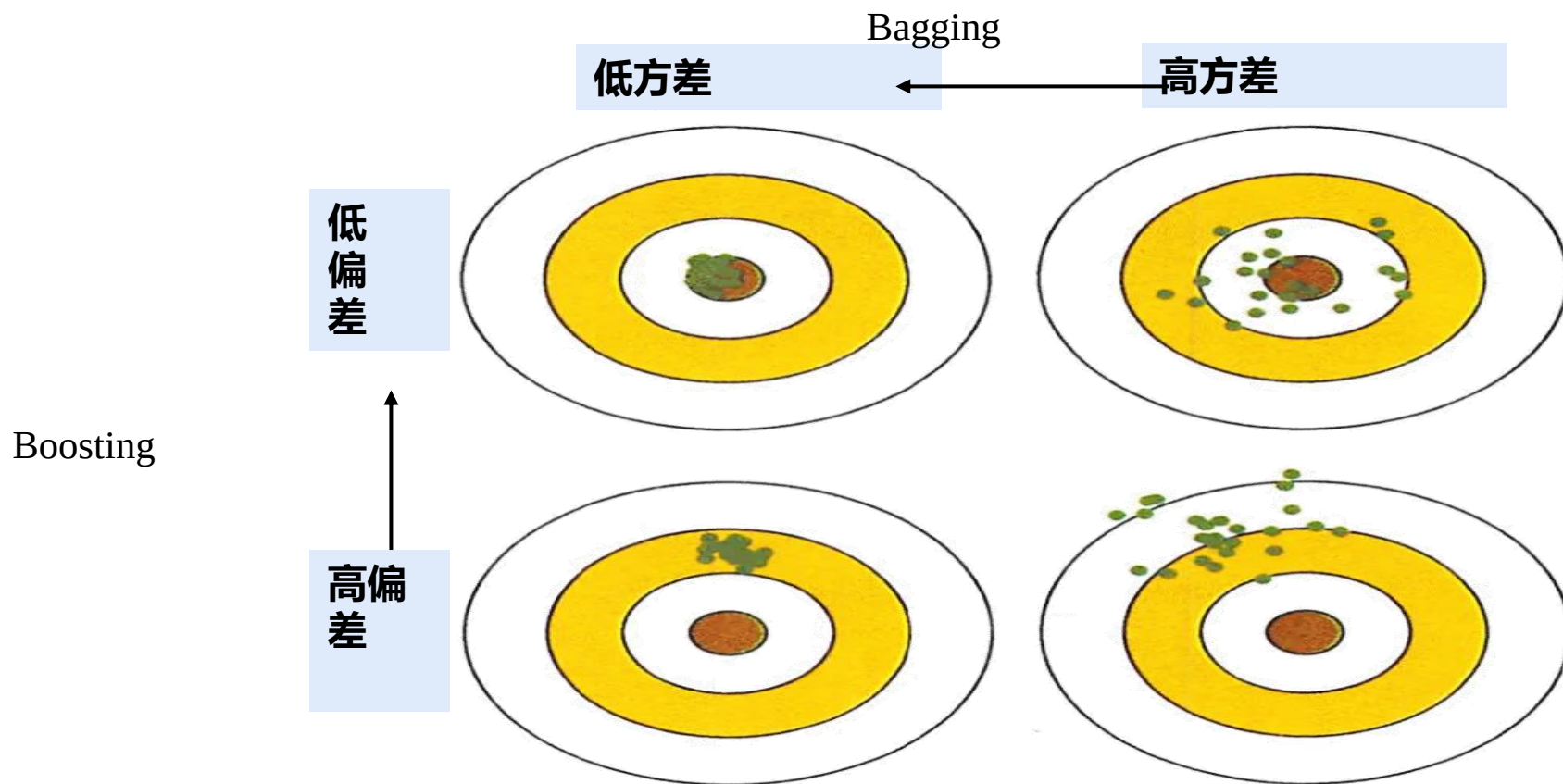
- 目前工业界使用最好的GBDT版本是**LightGBM** (Light Gradient Boosting Machine) , 它是微软于2017年提出的一个基于梯度提升框架的集成学习方法。LightGBM可视为GBDT的一个工业优化版本, 主要用于解决GBDT在海量数据中遇到的问题, 更好地应用于工业实践中



[扩展阅读: LightGBM - 最快的梯度提升模型](#)

4 Boosting方法与偏差

- 从偏差-方差角度来看，Boosting主要**关注降低偏差**。因此Boosting能基于泛化性能弱的学习器构建出很强的集成





武汉大学
WUHAN UNIVERSITY

03 Bagging与随机森林

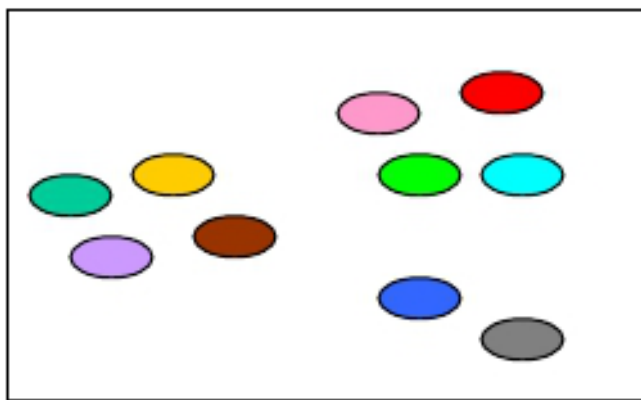
Bagging and Random Forest



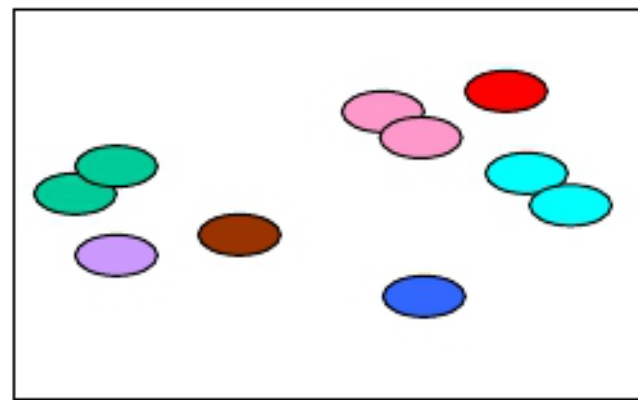
- Bagging (Bootstrap Aggregating) 是一类**并行**式集成学习方法（相同算法并行生成个体学习器）
- Bagging旨在通过组合多个弱学习器来提高整体模型的**预测性能**和**鲁棒性**，其核心思想是通过**结合多个独立训练的模型**来降低预测**方差**，提高模型的**稳定性**

1 样本扰动与平衡

- 其基于**自助采样法** (bootstrap sampling) 给定包含 m 个样本的数据集 D , 每次随机从 D 中挑选一个样本, 将其副本放入 D' (该样本仍在数据集 D 中, 使得下次采样时仍有可能被选中)。重复执行 m 次, 就可以得到了包含 m 个样本的数据集 D'



训练集与原样本集同规模

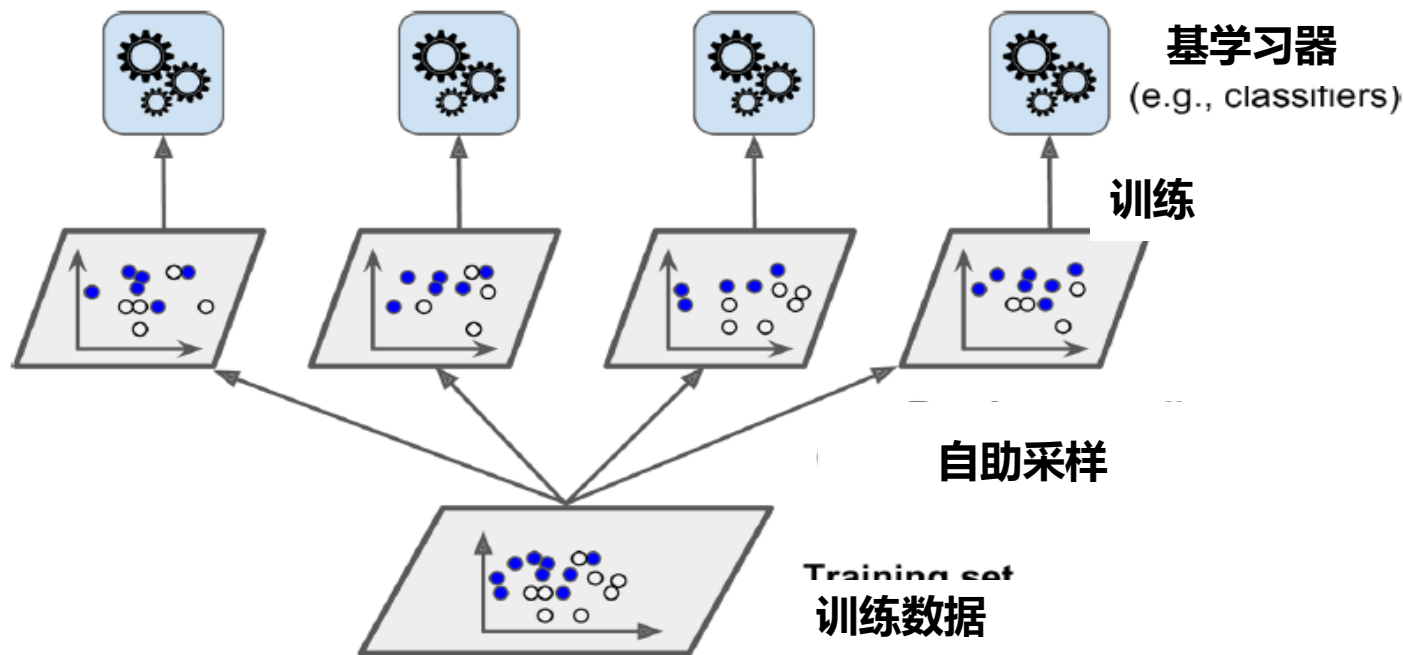


数据分布有所改变

*在 m 次采样中, 某样本始终不被选中的概率极限

$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m \mapsto \frac{1}{e} \approx 0.368$$

- 照此来得到 T 个采样集，然后基于采样集来训练基学习器，最终使用**投票**或**平均**策略将这些学习器组合



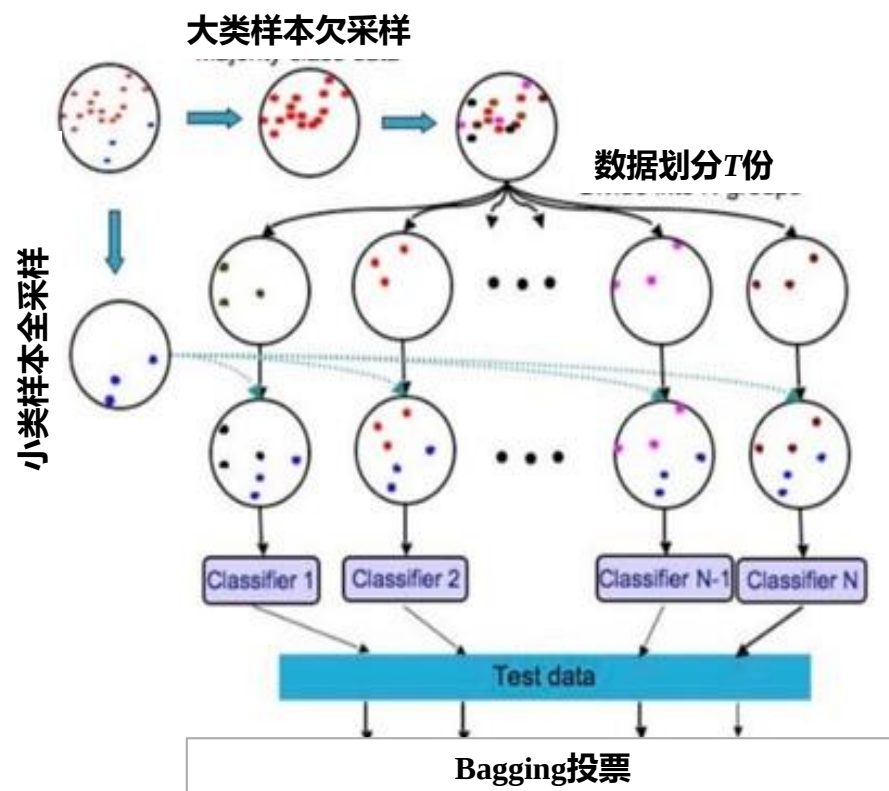
Bagging集成学习流程

Bagging算法高效且能直接应用与分类（简单投票）和回归任务（简单平均）。自助采样法剩下大约36.8%的样本来对泛化性能进行“**包外估计**”（out-of-bag estimate）

- 机器学习中经常会遇到数据标签 (label) **不平衡**的样本。如果不做任何处理就直接训练模型，模型效果一般不是很理想
- 可采用**随机欠采样**等方法构造**T份**样本并训练T个基学习器，再用Bagging方法集成，这种方法称为**EasyEnsemble**



e.g 建立一套银行反洗钱模型。在银行业务里，有洗钱嫌疑的转账毕竟是少数，如果有洗钱嫌疑的标记为1，安全的标记为0，那么这标签1:0的比率通常会在100以上，这就是标签不平衡





2 平均法

- 对于**回归**任务，学习器输出 $g_t(\mathbf{x}) \in R$ 为数值型，平均法组合策略是对T个学习器输出**取平均**

$$G(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T g_t(\mathbf{x})$$

$$G(\mathbf{x}) = \sum_{t=1}^T w_t g_t(\mathbf{x})$$

注意这个权重

其中 w_t 是个体学习器 g_t 的权重，通常要求 $w_t \geq 0$ ， $\sum_{t=1}^T w_t = 1$

3 投票法

- 对于**分类**任务，学习器 g_t 从类标记集合 $\{c_1, c_2, \dots, c_k\}$ 中预测出一个标记，最常见的集成策略是**投票**。将 g_t 在样本 x 上的预测输出（**分类概率向量**）表示为一个 k 维向量 $[g_t^1(x), g_t^2(x), \dots, g_t^k(x)]^T$ ，其中 $g_t^j(x)$ 是 g_t 预测的 x 属于第 c_j 类的概率或类标



e.g. 假设创建了一个包含1000个分类器的集成学习器，每个分类器都只有51% 的正确率（几乎没比随机猜测强多少）。如果以简单多数投票的类别作为预测结果，可以期待的准确率高达75%

$$\sum_{k=501}^{1000} C_{1000}^k p^k (1-p)^{1000-k} \approx 0.75$$

- **绝对多数**投票法 (majority voting)

即若某标记得票过半数，则预测为该标记；否则拒绝预测

- **相对多数**投票法 (plurality voting)

预测为票最多的标记，若同票则随机选取一个

- **加权**投票法 (weighted voting)

$$G(\mathbf{x}) = c_{\arg \max_j \sum_{t=1}^T w_t g_t^j(\mathbf{x})}$$

再次注意这个权重

其中 w_t 是个体学习器 g_t 的权重，通常要求 $w_t \geq 0$ ， $\sum_{t=1}^T w_t = 1$



- 现实任务中可能产生，不同类型个体学习器可能产生不同类型的 g_t^j ：
 - **类标记**： $g_t^j \in \{0,1\}$ ，输出0或1，亦被称为“**硬投票**”。
 - **类概率**： $g_t^j \in [0,1]$ ，输出一个后验概率，亦被称为“**软投票**”

不同类型的 g_t^j 值**不能混用**

- 投票法集成希望所有的分类器都是**完全独立**的。显然在实际工作中这是不可能的，因为这些学习器都是在相同的数据集下训练产生，它们很可能会犯相同的错误，导致集成的准确率有所降低

- Bagging是基于多个自助样本集，训练多个**大同小异**的基学习器，然后通过**简单多数投票法**对结果集成，得到性能得到提升的集成分类器（对回归学习器，则采用**简单算术平均法**进行结果集成）。

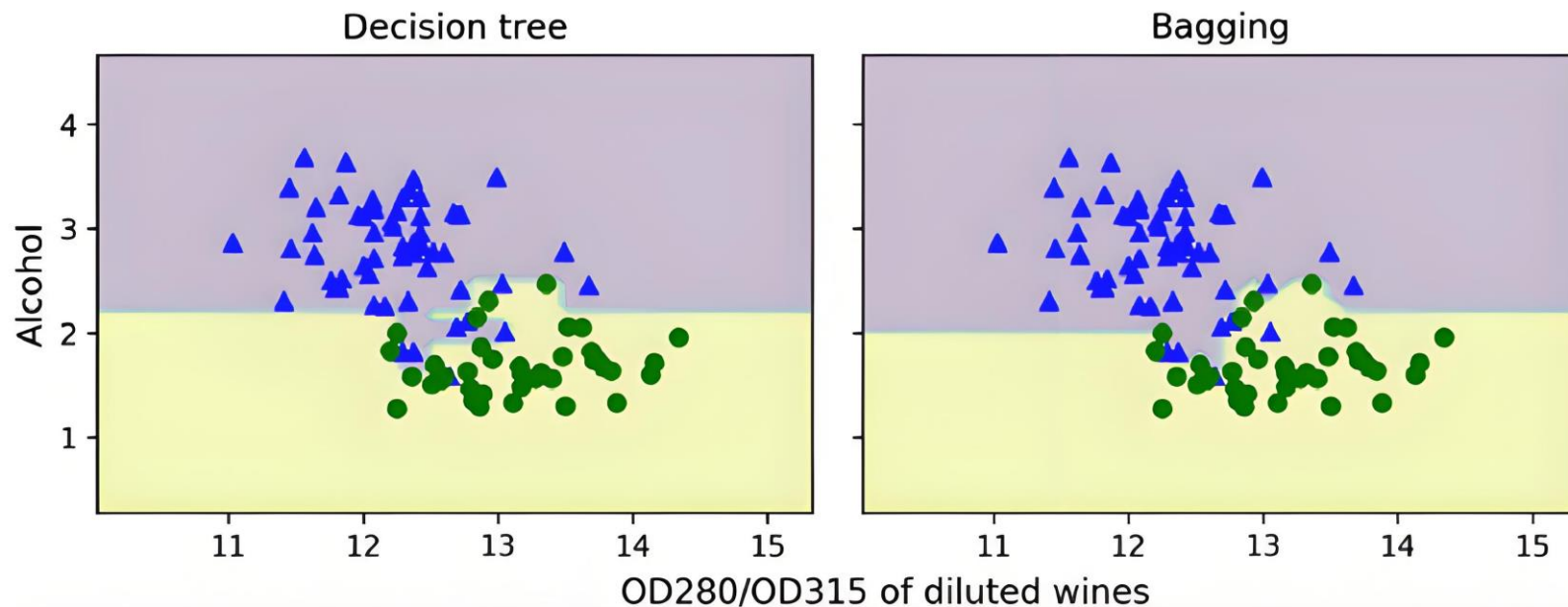
输入：训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程:

1: **for** $t = 1, 2, \dots, T$ **do**
2: $h_t = \mathcal{L}(D, \mathcal{D}_{bs})$
3: **end for**

输出: $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = y)$

自助聚合 (Bagging) 算法流程图



```
>>> from sklearn.ensemble import BaggingClassifier
>>> tree = DecisionTreeClassifier(criterion='entropy',
...                               random_state=1,
...                               max_depth=None)
>>> bag = BaggingClassifier(base_estimator=tree,
...                           n_estimators=500,
...                           max_samples=1.0,
...                           max_features=1.0,
...                           bootstrap=True,
...                           bootstrap_features=False,
...                           n_jobs=1,
...                           random_state=1)
```



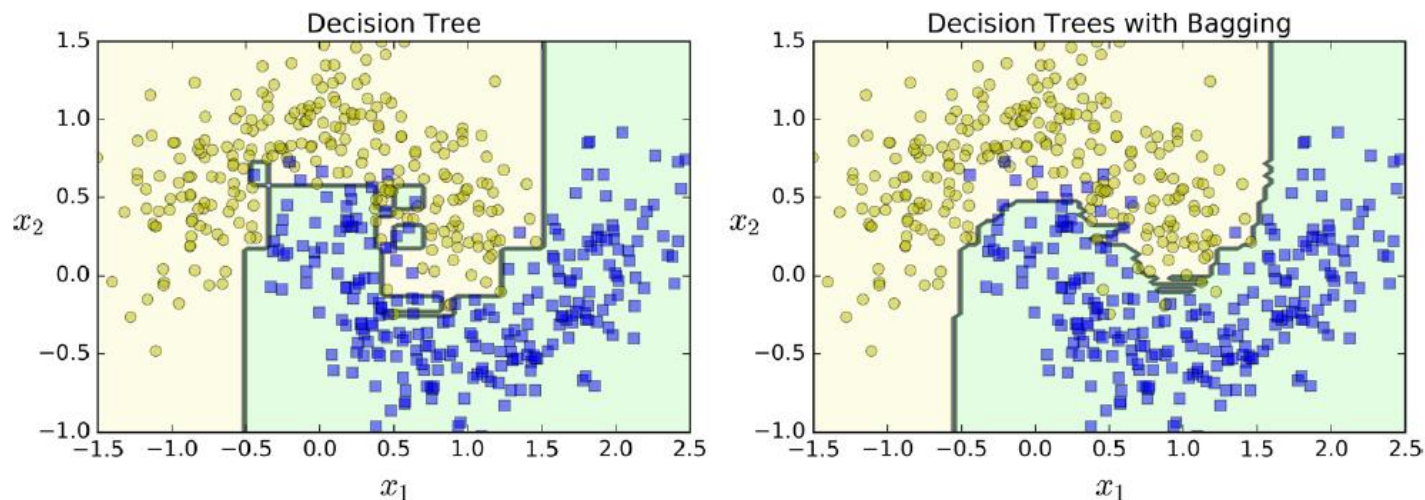
```
>>> from sklearn.metrics import accuracy_score
>>> tree = tree.fit(X_train, y_train)
>>> y_train_pred = tree.predict(X_train)
>>> y_test_pred = tree.predict(X_test)
>>> tree_train = accuracy_score(y_train, y_train_pred)
>>> tree_test = accuracy_score(y_test, y_test_pred)
>>> print('Decision tree train/test accuracies %.3f/%.3f'
...       % (tree_train, tree_test))
Decision tree train/test accuracies 1.000/0.833
```

修剪的决策树正确地预测了所有训练样本的分类标签;
然而, 测试准确率很低 (0.833), 表明决策树存在高方差 (过拟合)

```
>>> bag = bag.fit(X_train, y_train)
>>> y_train_pred = bag.predict(X_train)
>>> y_test_pred = bag.predict(X_test)
>>> bag_train = accuracy_score(y_train, y_train_pred)
>>> bag_test = accuracy_score(y_test, y_test_pred)
>>> print('Bagging train/test accuracies %.3f/%.3f'
...       % (bag_train, bag_test))
Bagging train/test accuracies 1.000/0.917
```

Bagging分类器与决策树的训练准确度相似 (都是100%),
但是, Bagging分类器在测试集上的泛化性能略好 (0.917)

- Bagging的结果聚合：每个预测器单独的偏差都高于在原始训练集上训练的偏差，但是通过聚合，可以在一定程度上同时降低了偏差和方差。聚合学习器与单个学习器相比，集成的**偏差相近**，但是**方差更低**。



单颗决策树与500 个决策树的bagging 集成对比

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=True, n_jobs=-1
)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```



c.g 两个学习器 g_1 和 g_2 各自预测一个4分类问题，输入样本 $\mathbf{x}$ ，得到分类后验概率如下。两个学习器的权重分别为0.7和0.3，则加权投票后的为：

	$c_1 (j=1)$	c_2	c_3	c_4
g_1^j	0.2	0.3	0	0.5
g_2^j	0.1	0.6	0	0.3
$\sum_{t=1}^2 w_t g_t^j(\mathbf{x})$	0.17	0.39	0	0.44

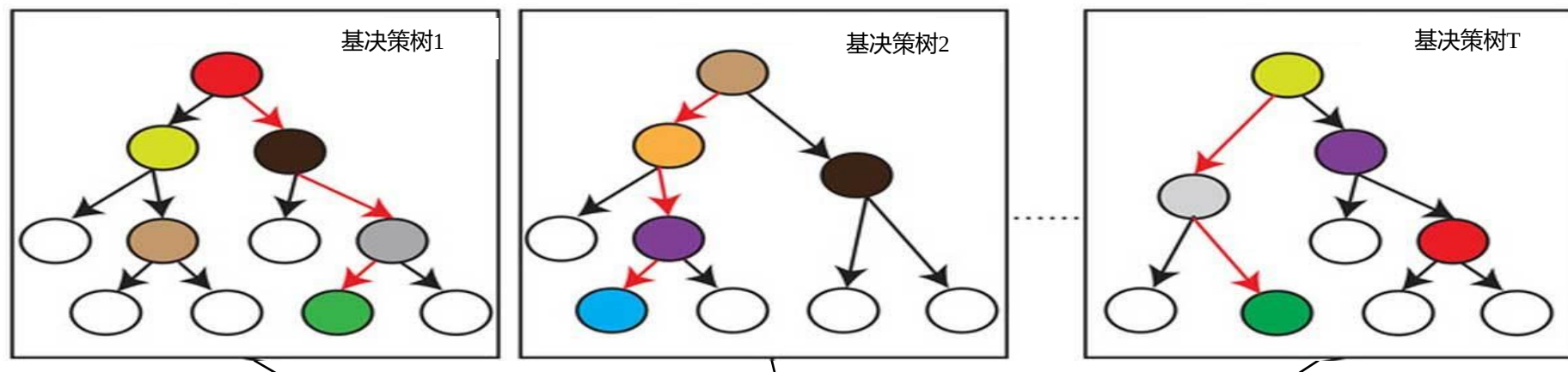
$$w_1 = 0.7$$

$$w_2 = 0.3$$

$$G(\mathbf{x}) = c_4$$

4 随机森林

- 以**决策树**方法构建基学习器的**Bagging**称为**随机森林** (Random Forrest, RF) , 是一种非常经典的机器学习方法



- 在**随机**自助采样的 m 个样本上训练决策树
- 在每个结点对应的 d 个特征属性上**随机**选 k 个特征, 进行最优划分

Bagging投票或平均

随机森林的计算流程



➤ 传统决策树在选择划分属性时是在当前结点的特征（属性）集合（假设 d 个）中选择一个。

而在随机森林方法中，在决策树的训练过程中引入了**随机属性选择**

➤ 对基决策树的每个结点，**先从该结点的特征集合中随机选择一个包含 k 个属性的子集**，然

后再从这个子集中选择一个最优属性用于划分。这里的参数 k 控制了随机性的引入程度：

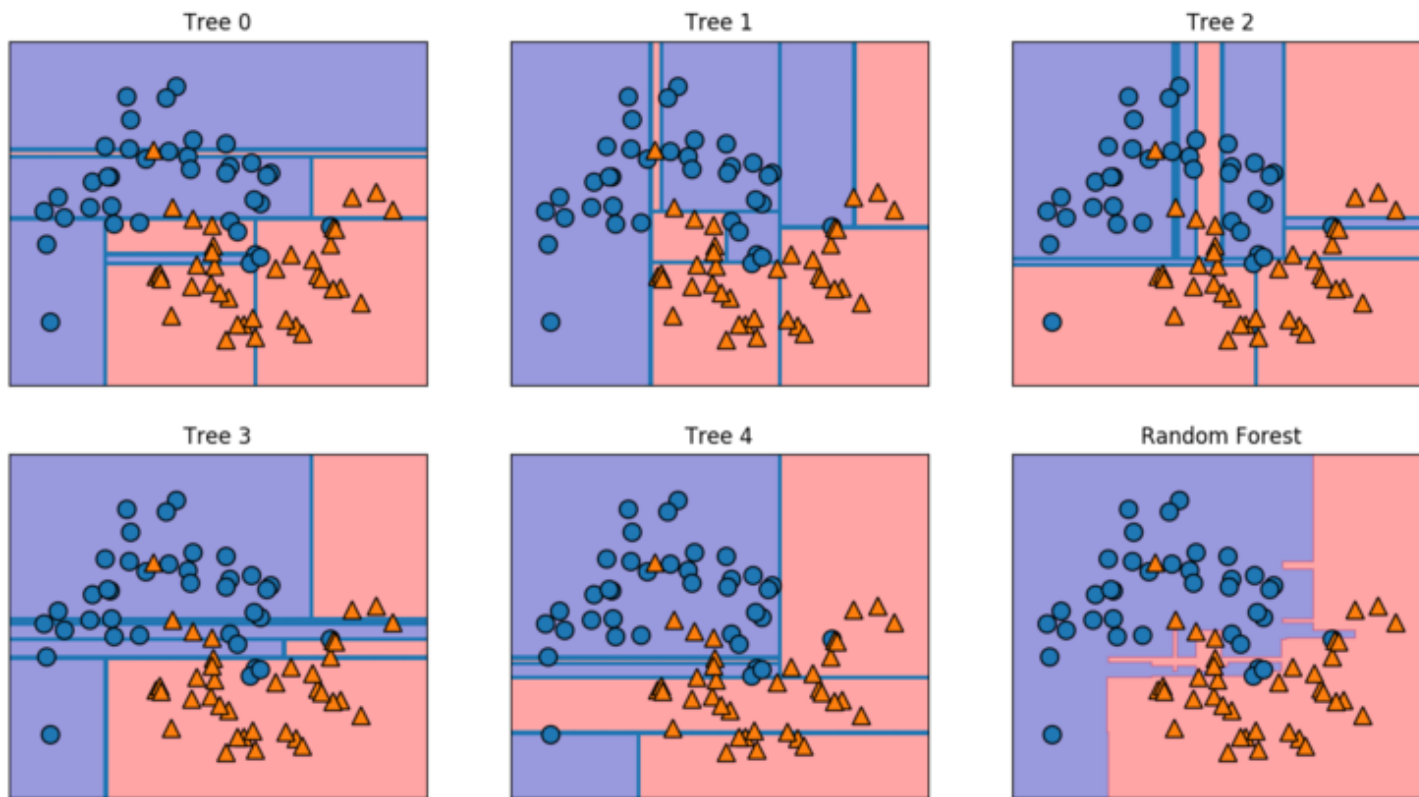
若令 $k = d$ ，则基决策树的构建与传统决策树相同；若令 $k = 1$ ，则是随机选择一个属性用

于划分；一般情况下， $k = \log_2 d + 1$



- RF与一般Bagging方法中基学习器的多样性主要来自样本扰动（自助采样）不同，RF中基学习器的多样性不仅来自**样本扰动**，还来自**属性扰动**，这就使得**个体学习器之间差异性得以增加**
- RF的解释性不如决策树，但不必为如何选择好的超参数而费心，且通常不需要剪枝。因为集成后的模型对来自于单个决策树的噪声抵抗力强。唯一需要关心的参数是随机森林中**决策树的个数**。通常情况下，树越多，随机森林分类器的性能就越好，伴随着计算成本就越大

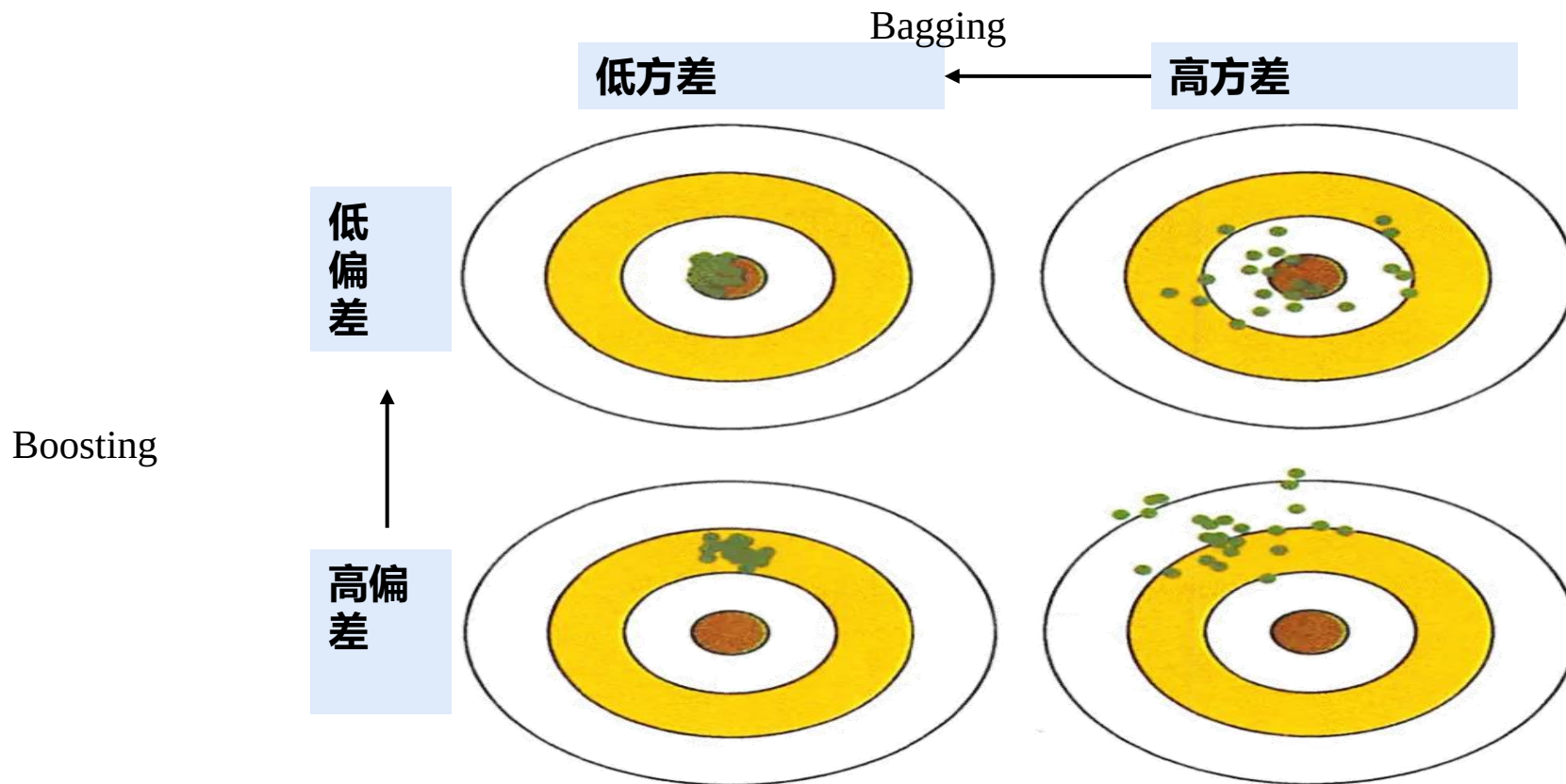
- 随机森林方法简单、易实现、计算开销小，在很多实际任务中展现出强大的性能，被誉为“**代表集成学习技术水平的方法**”



随机森林分类的效果示例

5 Bagging方法与方差

- 从偏差-方差角度来看，Bagging主要**关注降低方差**，因此它在不剪枝决策、神经网络等易受**样本扰动**的学习器上效用更为明显





- **Stacking**将多个不同模型的预测结果作为输入，训练一个元模型（次级学习器）来生成最终的预测结果。Stacking允许使用**不同类型**的基础模型进行组合，从而提高了模型的多样性
- Stacking在多种机器学习竞赛中取得了优异的成绩，特别是在**处理复杂问题**时表现极为出色。如何更有效地**选择基础模型和元模型**以及如何**优化训练**过程是Stacking主要研究的问题

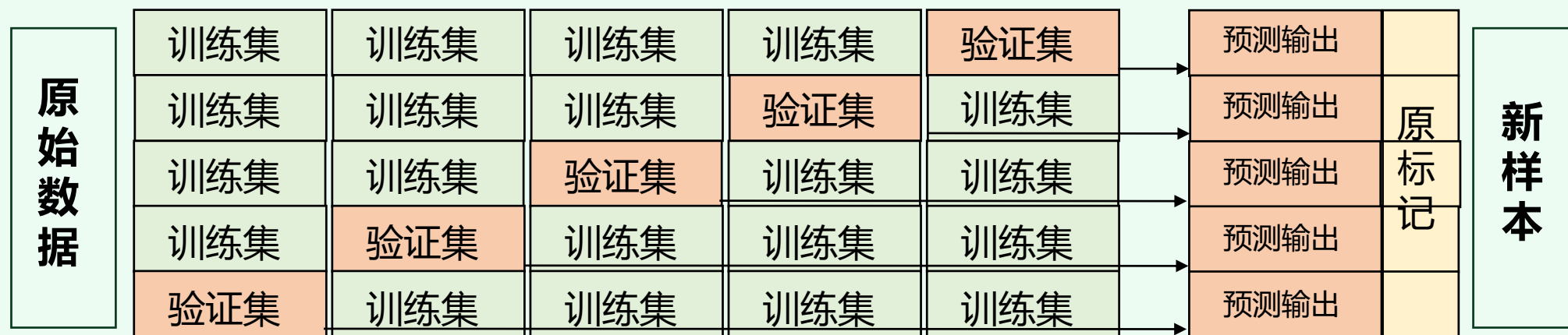
1 堆叠集成举例

- Stacking首先在原始数据中训练初级学习器，然后生成一个“**新数据集**”训练次级学习器。一般在这个“新数据集”中，初级学习器的输出被当作样例特征，原始样本的标记仍被当作样例标记



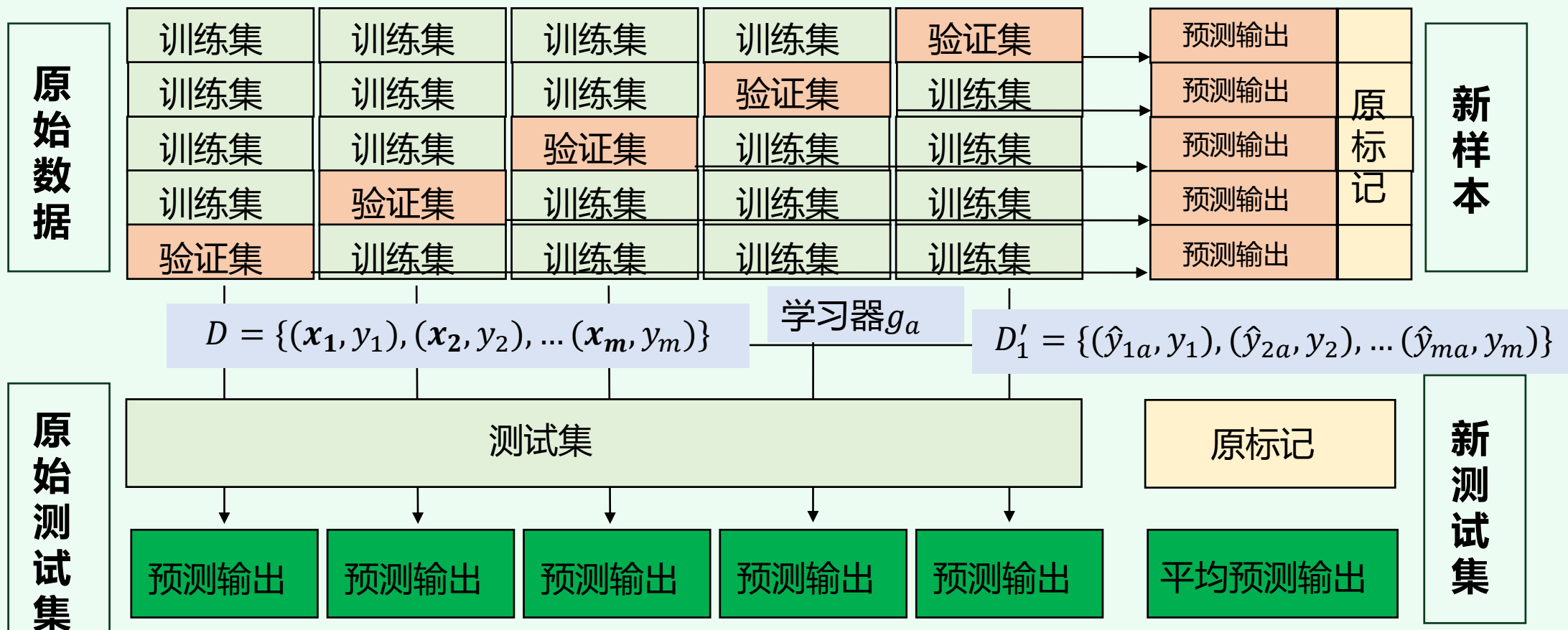
eg 用初级学习器在训练集上进行5折交叉验证，最终输出5个验证集的预测，把它们按行拼接起来

4份训练，1份验证



$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\} \xrightarrow{\text{学习器 } g_a} D'_a = \{(\hat{y}_{1a}, y_1), (\hat{y}_{2a}, y_2), \dots, (\hat{y}_{ma}, y_m)\}$$

- 用训练好的模型对**测试集**数据进行预测，输出的5个预测值取平均作为测试集的预测输出



- 假设有 a 、 b 、 c 三个初级学习器，那么经过上面的操作后，得到每个学习器对应的**新样本**（包含训练集、标记值和测试集）
- 将三个初级学习器得到的新样例特征和原标记构成新样本。一般使用**LR模型**作为次级学习器进行训练

提问：次级学习器训练了什么？

新样本集				
	新特征1	新特征2	新特征3	原标记

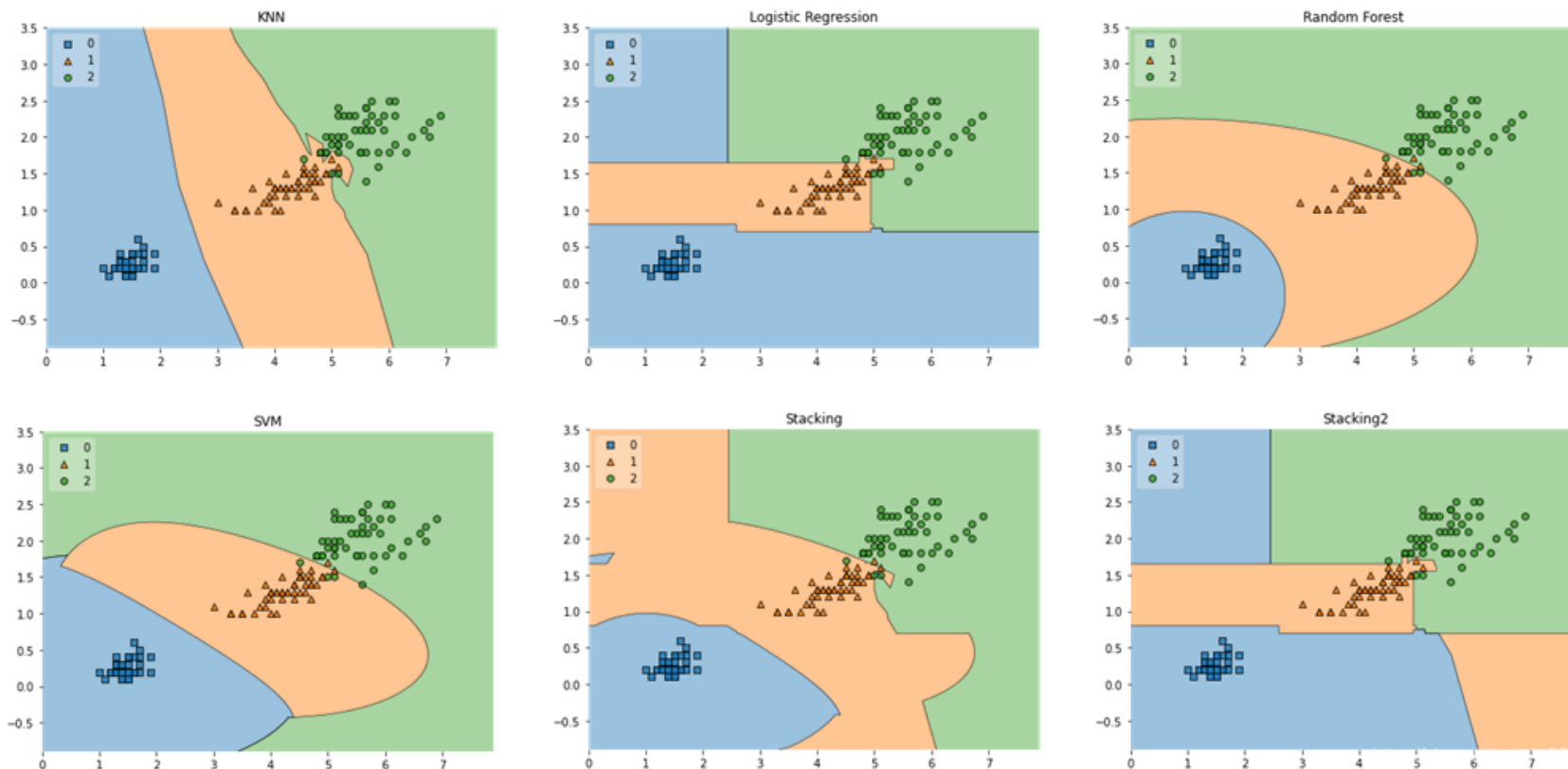
$$D' = \{([\hat{y}_{1a}, \hat{y}_{1b}, \hat{y}_{1c}], y_1), ([\hat{y}_{2a}, \hat{y}_{2b}, \hat{y}_{2c}], y_2), \dots ([\hat{y}_{ma}, \hat{y}_{mb}, \hat{y}_{mc}], y_m)\}$$

新测试集	平均预测输出 a	平均预测输出 b	平均预测输出 c	

$$G(x) = G(g_a(x), g_b(x), g_c(x))$$

2 堆叠性能示例

- Stacking不是把模型的结果组织起来，而是把**模型组织起来**。通过次学习器“学习”到各个基学习器的权重



Stacking分类的效果示例



武汉大学
WUHAN UNIVERSITY

第四节 多样性



➤ 定义学习器 h_i 的分歧 (ambiguity)

$$A(h_i | \mathbf{x}) = (h_i(\mathbf{x}) - H(\mathbf{x}))^2$$

➤ 集成的分歧

$$\begin{aligned}\bar{A}(h | \mathbf{x}) &= \sum_{i=1}^T w_i A(h_i | \mathbf{x}) \\ &= \sum_{i=1}^T w_i (h_i(\mathbf{x}) - H(\mathbf{x}))^2\end{aligned}$$



- 分歧项代表了个体学习器在样本 $\mathbf{x}$ 上的不一致性，即在一定程度上反映了**个体学习器的多样性**，个体学习器 h_i 和集成 H 的**平方误差**分别为

$$E(h_i | \mathbf{x}) = (f(\mathbf{x}) - h_i(\mathbf{x}))^2$$

$$E(H | \mathbf{x}) = (f(\mathbf{x}) - H(\mathbf{x}))^2$$

- 令 $\bar{E}(h | \mathbf{x}) = \sum_{i=1}^T w_i \cdot E(h_i | \mathbf{x})$ 表示**个体学习器误差的加权均值**

$$\begin{aligned}\bar{A}(h | \mathbf{x}) &= \sum_{i=1}^T w_i E(h_i | \mathbf{x}) - E(H | \mathbf{x}) \\ &= \bar{E}(h | \mathbf{x}) - E(H | \mathbf{x}).\end{aligned}$$

➤ 个体学习器 h_i 在全样本上的泛化误差和分歧项分别为

$$E_i = \int E(h_i | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

$$A_i = \int A(h_i | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

➤ 集成的泛化误差为

$$E = \int E(H | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

➤ 令 $\bar{E} = \sum_{i=1}^T w_i E_i$ 表示个体学习器泛化误差的**加权均值**,

$\bar{A} = \sum_{i=1}^T w_i A_i$ 表示个体学习器的**加权分歧值**, 有

$$E = \bar{E} - \bar{A}$$



- 上式表明：个体学习器精确性越高、多样性越大，则集成效果越好，称为误差-分歧分解
- 为什么不能直接把 $\bar{E} - \bar{A}$ 作为优化目标来求解？
- 因为现实任务中很难直接对 $\bar{E} - \bar{A}$ 进行优化：
 - 它们定义在整个样本空间上
 - $\bar{A}$ 不是一个可直接操作的多样性度量
 - 上面的推导过程**只适用于回归学习**，难以直接推广到分类学习任务上去

- 多样性度量 (diversity measure) 用于度量集成中个体学习器的多样性
- 对于二分类问题, 分类器 h_i 与 h_j 的预测结果联立表 (contingency table) 为

	$h_i = +1$	$h_i = -1$
$h_j = +1$	a	c
$h_j = -1$	b	d

$$a + b + c + d = m$$

➤ 常见的多样性度量

- 不合度量 (Disagreement Measure)

$$dis_{ij} = \frac{b + c}{m}$$

- 相关系数 (Correlation Coefficient)

$$\rho_{ij} = \frac{ad - bc}{\sqrt{(a + b)(a + c)(c + d)(b + d)}}$$

- Q -统计量 (Q-Statistic)

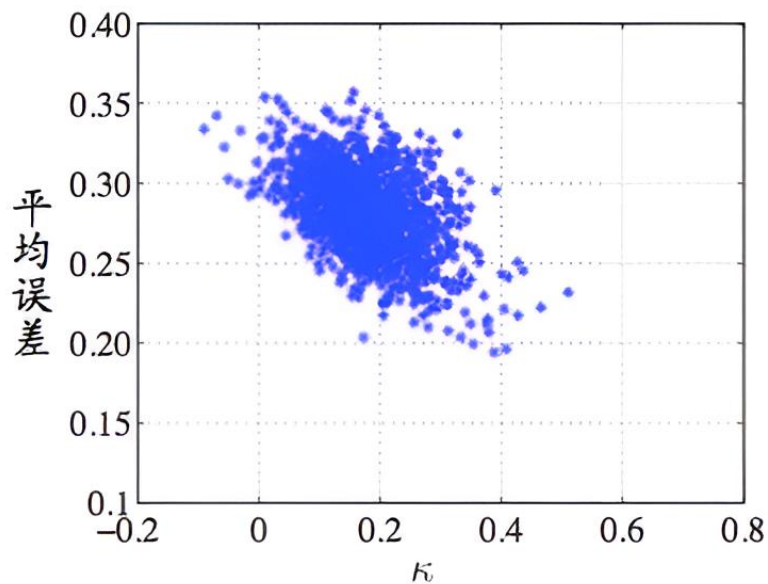
$$Q_{ij} = \frac{ad - bc}{ad + bc} \quad |Q_{ij}| \leq |\rho_{ij}|$$



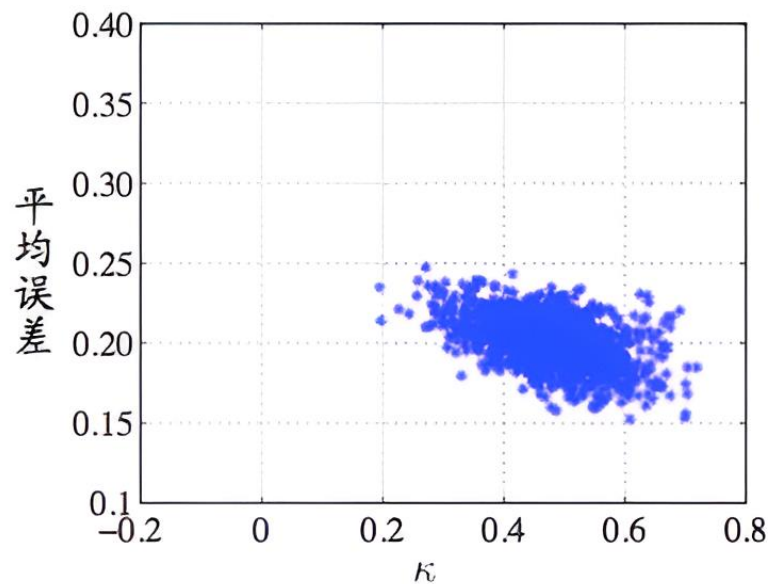
➤ κ -统计量 (Kappa-Statistic)

$$\kappa = \frac{p_1 - p_2}{1 - p_2}$$
$$p_1 = \frac{a + d}{m},$$
$$p_2 = \frac{(a + b)(a + c) + (c + d)(b + d)}{m^2}$$

- 若分类器 h_i 和 h_j 在数据集上完全一致, 则 $\kappa = 1$; 若它们仅是偶然达成一致, 则 $\kappa = 0$; κ 通常取非负值, 仅在两个分类器达成一致的的概率低于偶然性的情况下取负值

➤ k -误差图

(a) AdaBoost 集成



(b) Bagging 集成

**点云的位置越高，个体分类器的准确性越低；
点云的位置越靠右，个体学习器的多样性越小。**



➤ 常见的增强个体学习器的多样性的方法

- 数据样本扰动
- 输入属性扰动
- 输出表示扰动
- 算法参数扰动

➤ 数据样本扰动通常是基于采样法

- Bagging中的自助采样法
- Adaboost中的序列采样



➤ 对数据样本的扰动敏感的基学习器（不稳定基学习器）

- 决策树，神经网络等

➤ 对数据样本的扰动不敏感的基学习器（稳定基学习器）

- 线性学习器，支持向量机，朴素贝叶斯，k近邻等

➤ 输出表示扰动

- 翻转法（Flipping Output）
- 输出调剂法（Output Smearing）
- ECOC法（输出纠错码）

➤ 随机子空间算法 (输入属性扰动) (random subspace)

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
基学习器数 T ;
子空间属性数 d' .

过程:

```
1: for  $t = 1, 2, \dots, T$  do  
2:    $\mathcal{F}_t = \text{RS}(D, d')$   
3:    $D_t = \text{Map}_{\mathcal{F}_t}(D)$   
4:    $h_t = \mathcal{L}(D_t)$   
5: end for
```

输出: $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\text{Map}_{\mathcal{F}_t}(\mathbf{x})) = y)$



➤ 算法参数扰动

- 负相关法（正则化）
- 不同的多样性增强机制同时使用

➤ 不同的多样性增强机制可同时使用，例如随机森林中同时使用了数据样本扰动和输入属性扰动，有些方法甚至同时使用了更多机制

- **提升** (Boosting): 级联式训练个体学习器, 每个后继学习器都对其前驱学习器做出一些改进。因此个体学习器之间存在强依赖关系, 必须串行生成
- **装袋** (Bagging): 每个个体学习器使用的算法是相同的, 但是在不同的样本集上进行训练。因此个体学习器之间不存在强的计算依赖关系, 可并行学习, 最后再进行结果集成
- **堆叠** (Stacking): 初级学习器并行运行, 级联次级学习器训练初级学习器权重, 兼有Bagging和Boosting的特点



请同学们自行从训练集划分、样本权重、弱学习器权重、泛化误差等方面, 对比 Boosting、Stacking和Bagging

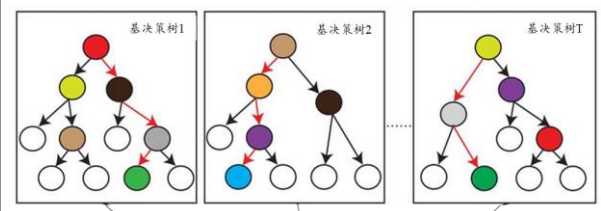


比较和总结



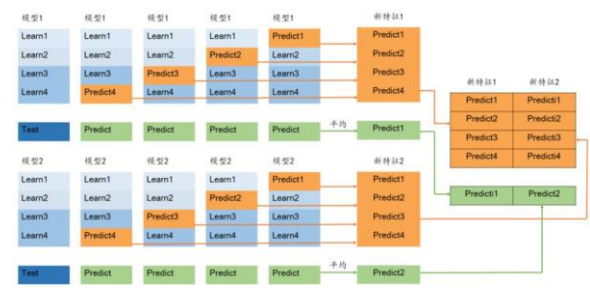
➤ 目前主流集成学习的方法:

Bagging/随机森林



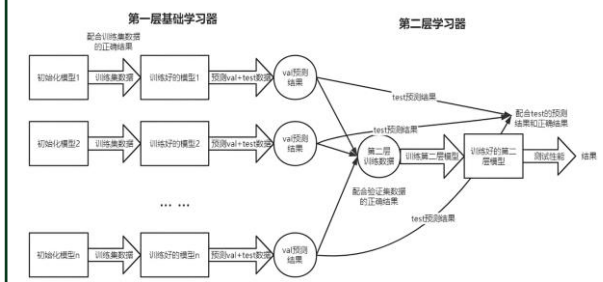
- 在**随机**自助采样的 m 个样本上训练决策树
- 在每个结点对应的 d 个特征属性上**随机**选 k 个特征, 进行最优划分
- Bagging投票或平均
- 最常见的集成学习

Stacking



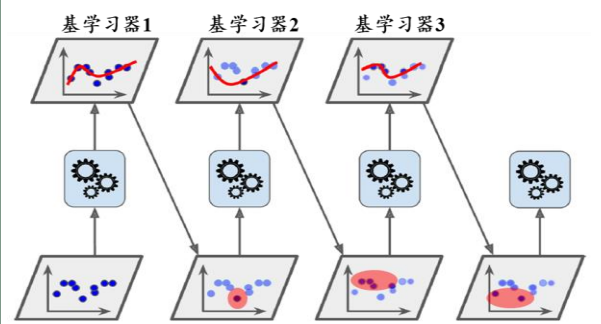
- 初级学习器/次级学习器

Blending (自学)



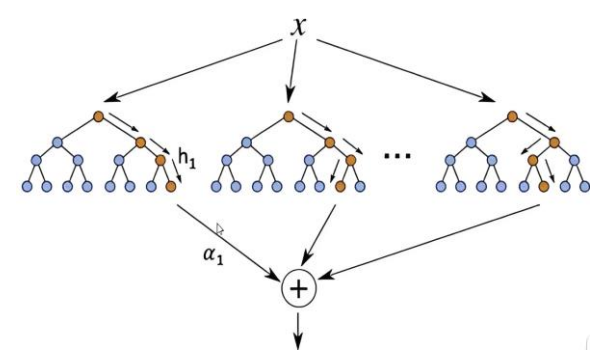
- 和Stacking类似

AdaBoost



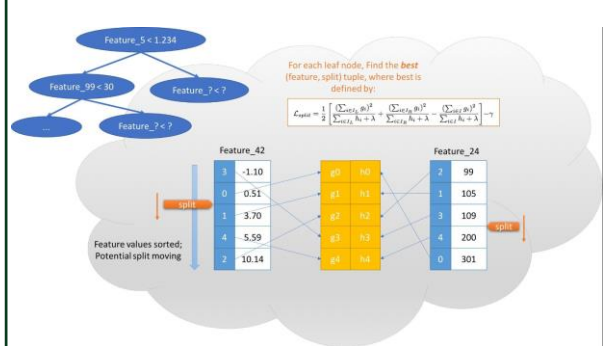
- 样本权重自适应更新

GBDT/CatBoost/GBM



- 拟合梯度优化决策树

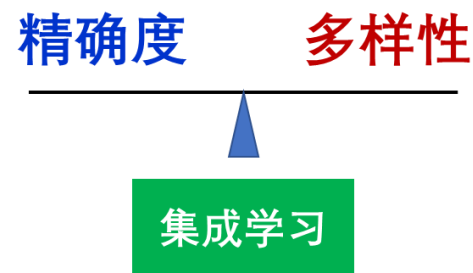
XGBoost



- 通用的梯度提升策略

- 集成多样性 (diversity)：集成**多样性**，即个体学习器之间的差异，是集成学习方法的重要议题
 - 若结合完全相同的个体学习器，显然不会有性能提升
 - 要想在结合之后有性能提升，个体学习器之间必须有差异
- 多样性至关重要，却难以刻画：目前还没有一个广泛接受的定义

理解多样性→集成学习
领域的圣杯问题



个体学习器应当兼具**精确性**与**多样性（好而不同）**。因为“互补性”比单纯的精度更重要，集成高精度的个体学习器不如结合多样性更好的个体学习器。**集成学习依赖于个体学习器精度和多样性之间的权衡**



建议阅读的网络文献



武汉大学
WUHAN UNIVERSITY

- [1]. Octopus. LightGBM 的完整解释：最快的梯度提升模型[EB/OL][2024-07-06]. <https://blog.csdn.net/zy345293721/article/details/134108203>
- [2]. 刘启林. XGBoost算法梳理[EB/OL][2019-10-27]. <https://zhuanlan.zhihu.com/p/162001079>
- [3]. 刘启林. GBDT的原理、公式推导、Python实现、可视化和应用[EB/OL][2019-10-20]. https://zhuanlan.zhihu.com/p/280222403?utm_source=wechat_session&utm_id=0

课后阅读西瓜书第8章



武汉大学
WUHAN UNIVERSITY

End of This Part
